μGAN: A Mutation-Based Cost Optimal Adversarial Malware Generation Approach Against Evolving Android Malware Variants

Xiaojian Liu[a,*], Zilin Qin[a], Kehong Liu[b]

[a]College of Computer Science and Technology, Xi'an University of Science and Technology, Xi'an, Shaanxi, China, 710054
[b]School of Artificial Intelligence, Xidian University, Xi'an, Shaanxi, China, 710071

Abstract

Malware *detection* and *evasion* represent two opposing sides engaged in a relentless arms race—to evade stringent detection, Android malware has developed a variety of sophisticated evasion techniques, been continuously evolving and spawning new malware variants, which poses severe threats to the security of Android defense systems. To tackle this problem, adversarial training offers a promising approach to improving the resilience of detection against newly emerging malware variants. However, in the setting of Android malware detection, adversarial training still faces a critical challenge—how to craft *valid* and *meaningful* adversarial samples. This paper proposes a mutation-based adversarial malware generation approach, which attempts to introduce proper perturbations to the seed samples in order to enable them to successfully evade detection. To seek for such perturbations, we formulate the problem of crafting adversarial malware as a constrained combinatorial optimization problem—adversarial samples should evade detection while consuming minimal crafting efforts. For this problem, we devise a solution strategy, referred to as μGAN, which combines strengths of the Generative Adversarial Networks and the Simulated Annealing algorithm, to screen the optimal adversarial samples. Furthermore, we retrain an enhanced malware classifier by augmenting the dataset with the generated adversarial malware samples to improve the performance of detection against new malware variants. Extensive experimental evaluation shows that, introducing perturbations into malware can significantly promote the ability of malware to evade security detection; the enhanced malware detector retrained using our approach demonstrates superior performance over other state-of-the-art classifiers.

Keywords: Android malware detection, Adversarial samples generation, Program mutations, Simulated Annealing, Constrained combinatorial optimization

1. Introduction

To protect users from threats of malicious software, Android system has developed a suite of comprehensive security defense mechanisms, such as *sandboxing* (Spreitzenbarth et al., 2013), *permissions* (i.e., *access controls* build in Android system) (Shrivastava et al., 2020), *App security vetting* (Chen et al., 2015), and *regular updates*. However, in order to effectively evade stringent defense systems, Android malware is engaged in an ongoing arms race with detection techniques, continuously evolving and spawning new malware variants, posing severe threats to the security of Android defense systems (360-Mobile-Security, 2024). Detecting Android malware, especially newly emerging and evolving malware variants, remains a significant challenge. In what follows, we first briefly review existing mainstream methodologies for Android malware detection, then highlight the challenges they confront.

Traditional approaches

Traditional Android malware detection methods, such as *signature*-based methods (Sihag et al., 2020), *anomaly analysis*-based methods (Sokolova et al., 2017), and *behavior*-based methods (Sun et al., 2018), need to extract specific features (statically or dynamically) from program codes (binary or source codes) or their execution traces, then detect malicious behaviors according to some *empirically derived* rules or patterns (Idika, 2007). As these rules or patterns are usually extracted from practical expertise or rich experience of security analysts, these methods usually perform well for *already known* malware (or *legacy* malware); but for newly emerging malware strains or zero-day attacks (Khalid et al., 2023), they often suffer from a significant shortfall in performance.

Machine Learning-based approaches

With the booming growth of Machine Learning (ML for short) techniques and the continuous accumulation of malware samples, many researchers introduced ML techniques to the field of malware detection (Kabakus, 2022; Singh and Tripathy, 2023; Li et al., 2024b). From the ML viewpoint, malware detection is treated as a task of *binary* or *multifamily* classification, which can be tackled by learning a high performance classifier from a large malware dataset (Liu et al., 2020). ML-based approaches focus on feature selection and extraction. The main features may involve *static features* (such as malware images (Vasan et al., 2020; Kim et al., 2018), raw bytes (Raff et al., 2018, 2021), opcodes (Seungho and Moon, 2020), and function calls (Huda et al., 2016)), *dynamic features* (such as API calls (Rabadi and

*Corresponding author
Email addresses: liuxiaojian@xust.edu.cn (Xiaojian Liu), 22208223057@stu.xust.edu.cn (Zilin Qin), kehongliu@stu.xidian.edu.cn (Kehong Liu)
URL: https://github.com/ansafe-zenith (Xiaojian Liu)

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Teo, 2020), network features (D. et al., 2019), behavior graphs (Wang et al., 2020; Han et al., 2021), and hardware features (Chen et al., 2018)), or a *hybrid* of the both (O'Shaughnessy and Sheridan, 2022; Yoo et al., 2021).

ML-based approaches outperform traditional methods by *automatically* identifying potential malicious behaviors from large malware datasets. This advantage reduces the demands for expert knowledge, thus offers a potential solution to detecting evolving malware by continuously learning from newly emerging malware samples (Zheng et al., 2024; Wang et al., 2022). However, ML-based approaches still encounter several challenges yet.

(1) *Data scarcity*. In certain scenarios, the datasets at hand are often insufficient for the training task, resulting in issues for the trained model, such as overfitting (Conti et al., 2022) or poor generalization (Xiong et al., 2024). For example, in the task of multifamily classifying, samples in some families, such as **AnserverBot**, **BeanBot**, and **SendPay**, are highly *sparse*, namely, only containing a few samples, yielding unsatisfactory classifying results.

(2) *Vulnerable to adversarial attacks*. Deep learning models are inherently susceptible to adversarial perturbations, leading to significant degradation in performance, even incorrect predictions (Huang et al., 2021). In particular, some attackers deliberately exploit this vulnerability and craft adversarial malware samples to deceive ML-based detectors (Suciu et al., 2019; Grosse et al., 2016).

(3) *Hard to keep up with rapid evolution of malware*. It is an ongoing *arms race* between defenders and attackers. To avoid being detected, attackers have developed a variety of sophisticated evasion techniques, such as code obfuscation (Rastogi et al., 2014; Maiorca et al., 2015), code encryption (Faruki et al., 2023), code polymorphism and metamorphism (Cani et al., 2014; Aydogan and Sen, 2015; Liu et al., 2019; Xu et al., 2016), and feature perturbation tree (He et al., 2023). Some attacks even covertly exploit zero-day vulnerabilities (Khalid et al., 2023) and side-channel strategies (Taylor et al., 2021) to launch attacks, making detection increasingly challenging.

Adversarial training

Adversarial training (Hu and Tan, 2022; Rebuffi et al., 2021; Szegedy et al., 2013) is widely recognized as a promising approach to improving the resilience of detection against newly emerging and evolving malware variants. The basic idea is to augment the dataset with *artificially crafted adversarial samples*, then *retrain* the classifier with the augmented datatset. However, in the context of Android malware detection, the major challenge of applying adversarial training is how to craft *valid* and *meaningful* adversarial samples. To clarify this challenge, let's introduce some help notations at first.

Let $\mathcal{M}$ be the malware sample space, $\boldsymbol{m} \in \mathcal{M}$ a piece of malware; $\boldsymbol{x} = \phi(\boldsymbol{m}) \in \mathcal{X}$ be the feature vector of $\boldsymbol{m}$, where ϕ is the feature extraction function, $\mathcal{X}$ is the feature vector space. Generating an adversarial sample, in essence, is to perform a perturbation $z : \mathcal{M} \to \mathcal{M}$ on sample $\boldsymbol{m}$ to produce a new one $\boldsymbol{m}' \in \mathcal{M}$.

Challenge 1. How to recover samples from perturbed feature vectors. Many research efforts (He et al., 2023; Xu et al., 2024) focus on generating adversarial samples $\boldsymbol{m}'$ by directly manipulating on the feature vectors $\boldsymbol{x} = \phi(\boldsymbol{m})$ of $\boldsymbol{m}$. As a common practice in image processing discipline, it may not yet be reasonable in crafting adversarial malware samples, because it is hard to reverse the perturbed feature vectors $\boldsymbol{x}'$ back to programs $\boldsymbol{m}' = \phi^{-1}(\boldsymbol{x}')$—the inverse relation ϕ^{-1} between feature vectors and program samples may become very complicated after complex vector transformations. Furthermore, we demand adversarial malware samples $\boldsymbol{m}'$ to maintain *compilable*, *installable*, *runnable*, and preserve *the functionality* of the original sample $\boldsymbol{m}$; this requirement imposes additional constraints on the generation of $\boldsymbol{m}'$ from $\boldsymbol{x}'$.

Challenge 2. How to retain dependency among features. Due to the strict structural specifications followed by Android programs, features of programs exhibit a strong mutual dependence; independently or arbitrarily modifying one or more features does not always yield valid adversarial samples. For example, if the Android API `getSystemService()` as a perturbation is added to the seed program, the required permissions `ACCESS_FINE_LOCATION` or `ACCESS_COARSE_LOCATION` must be accordingly added to the manifest file; otherwise the adversarial sample may not execute properly. It is thus essential to maintain consistency among features to ensure that the generated samples are valid and meaningful.

Challenge 3. How to effectively perform perturbations on samples. Various techniques have been proposed to perturb malware samples while preserving their functionality, such as adding (or injecting) bytes (Demetrio et al., 2021) to the code, inserting program blocks (Kucuk and Yan, 2020), and performing limited perturbation operations (Song et al., 2020). However, some of these perturbations are easily detected and disabled by defense methods (Pierazzi et al., 2020). Alternatively, *data augmentation* techniques (Hendrycks and Dietterich, 2019), a mainstay in the field of machine vision, have been increasingly applied to the creation of malware samples (Catak et al., 2021). The works (Yu et al., 2022; Dong et al., 2024) introduced a series of simple code refactoring rules (including *local variable renaming, duplication, and dead store elimination*) to metamorphose existing malware samples. Studies like MixCode (Gupta et al., 2017) and (Arya and Peddoju, 2024) have explored direct interpolation of mixed code features into programs to derive a variety of new variants. However, as reported in these studies, the performance of these simple methods is unsatisfactory, it is still demanded to develop more effective perturbation strategies to produce samples with strong evasion capabilities.

Our approach

To tackle these challenges, in this paper, we propose a mutation-based adversarial malware generation approach. An adversarial sample $\boldsymbol{m}'$ is created by introducing a proper perturbation z (a sequence of mutations) to the seed sample $\boldsymbol{m} \in \mathcal{M}$, i.e., $\boldsymbol{m}' = \boldsymbol{m} + z$. To find such a perturbation z, we formulate the problem of crafting adversarial malware samples as a *constrained combinatorial optimization* problem

$$\min_{z \in \mathcal{Z}^{\leq n}} cost(z) \ \text{ s.t. } c(z),$$

where $cost(z)$ is the crafting cost of z, $c(z)$ is a constrain set

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

must be satisfied. To solve this problem, we devise an optimization solution strategy, referred to as μGAN, that combines the Generative Adversarial Networks (GANs) (Goodfellow et al., 2016) and the Simulated Annealing (SA) algorithm (Pardalos and Mavridou, 2020), to seek for the optimal solution over the perturbation space $\mathcal{Z}^{\leq n}$.

The workflow of our approach consists of two phases, as illustrated in Figure 1. We firstly craft adversarial malware samples by solving the above optimization problem, then use the newly created adversarial samples to enrich the dataset and retrain the substitute detector to obtain an enhanced malware classifier.

Advantages of our approach

First, with the generated adversarial malware samples, the volume of the training dataset will be significantly augmented; it is crucial for the cases where available samples are severely inadequate for a training task. *Second*, the style of generating adversarial samples tends to simulate some potential variation patterns and evolving directions of malware, thus training with these samples will substantially enhance the power of the detector to identify newly emerging attacks. *Finally*, as the crafted samples are deliberately designed being *adversarial*, i.e., to escape from stringent detections, retraining with them will improve the precision as well as the robustness of the classifier.

Main contributions

The main contributions of this work are of what follows:

(1) *Mutation-based perturbations*. We formally define a perturbation as a sequence of *basic mutation operations* (referred to as *atomic mutations* in this paper) on a seed, and propose 17 atomic mutations for Android Apps, which cover typical code refactors, encrypting sensitive or personal data, reflections, obfuscations, and various semantically equivalent transformations. These mutations are explicitly imposed on Smali codes of Apps rather than feature vectors, eliminating the need for deriving the inverse function ϕ^{-1} (see *Challenge 1*). Moreover, we require these atomic mutations be *conservative*, i.e., they must maintain the correct syntax structures and functionalities of the seed programs, ensuring the generated adversarial samples to be compilable, installable, runnable, and semantics preserving (see *Challenge 2*).

(2) *Crafting adversarial samples with minimal edit efforts*. To address *Challenge 3*, we formulate the issue of generating adversarial malware samples as a constrained combinatorial optimization problem—adversarial samples should evade stringent detection while consuming minimal crafting efforts. To solve this optimization problem, we first transform it into an unconstrained optimization problem by using penalty function method, then employ μGAN strategy to find the optimal solution. Unlike the common practice for general combinatorial optimization problems (such as Traveling Salesman Problem), we devise a new neighborhood structure, called "1-bit-change & appending" for our solutions, and employ Metropolis Hasting algorithm to sample the next solution from the neighborhood. This treatment allows worse solutions to be accepted and explored with a certain likelihood, circumventing the pitfalls of local optima.

(3) *Full use of third-party detectors*. We view a collection of third-party detectors as a black box, called *reference detectors* $R(x)$, and train a *substitute detector* $F(x;\theta)$ to fit the performance of $R(x)$ closely. The insight behind this practice mirrors the wisdom of the proverb: "*The intelligence of the crowd surpasses that of a single sage*."—the principle that a group of detectors typically outperform a single one.

Extensive experimental evaluation shows that, μGAN can generate adversarial samples with a faster convergence and lower detection rate than existing leading approaches; 95% of the generated adversarial samples can successfully evade detection. The enhanced classifier obtained through adversarial training achieves a strong robustness in detecting against newly emerging and evolving malware variants, and exhibits a superior performance over the state-of-the-art malware detectors.

The remainder of this paper is organized as follows. Section 2 gives a motivating example to illustrate how to perturb a piece of malware. Section 3 formulates our problem and outlines our solution. Section 4 first addresses the main issues faced in our approach, then elaborates the implementations of these issues. The experimental results are reported in Section 5, and related work and conclusion are finally presented in Sections 6 and 7 respectively.

2. Motivating example

In this section, we give a motivating example to show how to introduce perturbations into a piece of malware step by step, so as to ultimately evade stringent detection successfully.

Listing 1: Example Java code of Yomi

```
// (a) Original code snippet of Yomi
void onCreate(Bundle state) {
  String dev = telMnger.getDeviceId();
  sms.sendTextMessage(num, null, dev, null,
      null);
}
// (b) Apply mutation "Indirect Method Call"
void onCreate(Bundle state) {
  String dev = telMnger.getDeviceId();
  indirectMethod(num, dev);
}
private void indirectMethod(String num, String
    dev) {
  sms.sendTextMessage(num, null, dev, null,
      null);
}
// (c) Apply mutation "Reflection"
void onCreate(Bundle state) {
  Method getDeviceIdMethod = TelephonyManager.
      class. getMethod("getDeviceId");
  String dev = (String) getDeviceIdMethod.
      invoke(telMnger);
}
// (d) Split a string into substrings
void onCreate(Bundle state) {
  String get = "get";
  String DeviceId = "DeviceId";
  Method getDeviceIdMethod = TelephonyManager.
      class. getMethod(get + DeviceId);
  String dev = (String) getDeviceIdMethod.
      invoke(telMnger);
}
```

This malware, called **Yomi**, belonging to malware family **Adrd**, poses a covert threat of *personal information leakages*, as

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

shown in Listing 1 (a). It collects device information by calling Android framework API `getDeviceId()` (called *source*-APIs) (at Line 3), and disseminates the information to the outside world by calling the API `sendTextMessage()` (called *sink*-API) (at Line 4). Note that, the following mutations are directly applied to the Smali code of **Yomi**, but for clarity, here we use Java source code for illustration.

This kind of threats can be discovered using a so-called *information flow* analysis method, such as FlowDroid (Arzt et al., 2014), which intends to detect an information propagation path from a source (i.e., `getDeviceId()`) to a sink (i.e., `sendTextMessage()`). This method works well only if two conditions are met: (1) source- and sink-APIs can be identified; (2) the data dependence relation between source and sink can be fully revealed. However, uses of Java reflection (Livshits et al., 2005) and Android ICCs (Inter-Component Communications), especially *implicit* ICCs (Octeau et al., 2015), may break the conditions, making information flow analysis challenging. We exactly exploit these mechanisms to perturb (or mutate) the program of **Yomi**, enabling it to evade detection successfully.

As illustrated in Listing 1 (b), (c), and (d), we perform a perturbation, i.e., a sequence of three mutations $z = \mu_1\mu_2\mu_3$ to **Yomi**:

- $m \xrightarrow{\mu_1} m_1$: *decompose* direct information flow `getDeviceId()` $\rightsquigarrow$`sendTextMessage()` into an indirect one via an intermediate method call `indirectMethod()` (at Lines 7-13);
- $m_1 \xrightarrow{\mu_2} m_2$: with the Java reflection mechanism, *modify* the hardcoded invocation of `getDeviceId()` to the reflective invocations (at Lines 15-18);
- $m_2 \xrightarrow{\mu_3} m_3$: to hide the source API's name, further *split* the whole method name "`getDeviceId`" into concatenated substrings "`get`" and "`DeviceId`" (at Lines 20-25). At this point, the perturbed malware m_3 is able to successfully evade detection of FlowDroid.

Let's explain the three mutations step by step. In the seed malware m, the information flow

`getDeviceId()` $\rightsquigarrow$ `sendTextMessage()`

occurs within the callback `onCreate()`, which can be easily detected by FlowDroid. To mislead detection mechanisms, we first try to decompose the information flow into two parts, `dev`$\rightsquigarrow$`indirectMethod(dev)` and `dev`$\rightsquigarrow$`sendTextMessage(dev)`, through an intermediate method call `indirectMethod()`. Unfortunately, the decomposed information flow can be still successfully detected by FlowDroid, as FlowDroid bases its analysis on the *IFDS* (Interprocedural Finite Distributive Subset) algorithm framework (Sagiv et al., 1996), which can precisely analyze interprocedural dataflows.

To evade the detection of FlowDroid, we further transform m_1 to m_2 through μ_2 by utilizing the Java reflection mechanism, i.e., modifying the hardcoded method invocation to the reflective one. However, FlowDroid is yet able to detect this information flow, because the name "`getDeviceId`" of the source API, appeared as a string constant, can be figured out by analyzing the reflection method `getMethod(.)`.

To hide the source API's name, we further split string "`getDeviceId`" as substrings "`get`" and "`DeviceId`", and concatenate them with "`+`" method of Java String class, as illustrated by μ_3. This time, FlowDroid fails to detect the information flow, because identifying the whole name "`getDeviceId`" requires the aid of a sophisticated string analysis tool (Wang et al., 2018), but unfortunately, not equipped with FlowDroid at its disposal.

In summary, what perturbation is introduced typically depends on the power of the target detector—the more powerful the detector, the more complex the introduced perturbation, and the greater the effort required to generate such perturbation. In this example, if we consider the number of mutations as a measurement of the cost for a perturbation z, i.e., $cost(z) = |z|$, and choose FlowDroid single as the target detector, then the evading cost of **Yomi** is 3 (i.e., $|z| = |\mu_1\mu_2\mu_3| = 3$). However, if we equip FlowDroid with a strong string solver, such as IC3 (Li et al., 2015), as the detector, then we must exert additional effort to create perturbations in order to bypass the detection.

3. Problem formulation & solution

In this section, we first formulate our problem (Section 3.1), then present our solution to this problem (Sections 3.2 and 3.3). To formulate our problem, we first introduce some notations, shown in Table 1.

Table 1: The notations

Notation	Meaning
$\mathcal{P}$	the domain of Android Apps. $P \in \mathcal{P}$ is an App.
$\mathcal{M} \subseteq \mathcal{P}$	the domain of malware samples. $m \in \mathcal{M}$ is a piece of malware.
$\mathcal{B} \subseteq \mathcal{P}$	the domain of benign samples.
$M \subseteq \mathcal{M}, B \subseteq \mathcal{B}$	the set of malware and benign samples used to train classifiers.
$\mathcal{Z}$	the finite set of atomic mutations. $\mu \in \mathcal{Z}$ is an atomic mutation.
$\mathcal{Z}^{\leq n}$	the set of perturbations with length no greater than n, n is the maximum permissible length of perturbations. $z \in \mathcal{Z}^{\leq n}$ is a perturbation.
$M_{adv} \subseteq \mathcal{M}$	the set of the generated adversarial malware.
$S \subseteq \mathcal{M}$	the set of malware seeds used to generate adversarial samples.
$F(x), F'(x)$	the substitute detector and the enhanced classifier. Given a threshold $\tau \in [0, 1]$, for a sample $x \in \mathcal{B} \cup \mathcal{M}$, if $F(x) > \tau$, x is classified as malware; otherwise, classified as benign.

3.1. Adversarial malware samples

A so-called *adversarial malware sample* is a type of malicious software that has been specifically designed or modified to evade detection by security systems. To formally specify this notion, we present the following definitions.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Definition 1. *(Perturbations) A perturbation $z : \mathcal{P} \to \mathcal{P}$ is a finite sequence of atomic mutations (i.e., basic mutation operations) $z = \mu_1\mu_2 \cdots \mu_k \in \mathcal{Z}^{\leq n}$ $(k \leq n)$, where $\mu_i \in \mathcal{Z} : \mathcal{P} \to \mathcal{P}$ is an atomic mutation that transforms an App $P \in \mathcal{P}$ to a new one.*

Definition 2. *(Application of perturbations) Given a seed malware $m \in \mathcal{M}$, we use a concise notation "$m + z$" to denote the application of perturbation z to m,*

$$m + z \triangleq z(m) \triangleq (\mu_k \circ \mu_{k-1} \circ \cdots \circ \mu_1)(m), \tag{1}$$

where $\mu_i \circ \mu_{i-1}$ is the function composition of μ_i and μ_{i-1}, i.e., $(\mu_i \circ \mu_{i-1})(m) = \mu_i(\mu_{i-1}(m))$. The perturbed version of m is denoted as m', i.e., $m' = m + z$.

Definition 3. *(Adversarial malware samples) An adversarial malware sample is such a specifically designed sample $m' = m + z$ that satisfies the following three constraints:*

1. ***C1**: $compilable(m') \wedge runnable(m')$. m' can be recompiled into APK files, and the recompiled APK files can be successfully installed and runnable in the smart phone or simulator.*
2. ***C2**: $m' \sim m \wedge m' \in \mathcal{M}$. m' is equivalent ("$\sim$" means the semantic equivalence relation) to m in terms of functionality, and keeps the maliciousness of m intact, i.e., $m' \in \mathcal{M}$.*
3. ***C3**: $F(m') \leq \tau$. m' tends to deceive the detector $F(x)$ into classifying it as benign, where $\tau \in [0, 1]$ is a given threshold.*

We usually expect to find an *optimal* adversarial malware sample m^* in terms of the *minimal crafting efforts*. This essentially equals to find a *minimal* perturbation z^* in the space $\mathcal{Z}^{\leq n}$, such that $m^* = m + z^*$ is a piece of adversarial malware.

A perturbation z can be measured by a given cost function $cost(z)$, which can be defined as a certain *edit distance* (Dolev et al., 2017) between m' and m, denoted as $d(m', m)$. Thus, the core of generating adversarial malware samples is to search for a perturbation z that transforms m to an adversarial malware $m' = m + z$ at the lowest cost. This problem can be formulated as the following constrained optimization problem

$$\begin{aligned} &\min_{z \in \mathcal{Z}^{\leq n}} cost(z) \\ &\text{s.t. } m' = m + z \\ &\quad compilable(m') \wedge runnable(m') \\ &\quad m' \sim m \wedge m' \in \mathcal{M} \\ &\quad F(m') \leq \tau, \end{aligned} \tag{2}$$

where $cost(z)$ is the objective function, z is the decision variable that subjects to the constraint set **C1**, **C2**, and **C3**.

To solve Problem 2, with the penalty function method (Fletcher, 1975), we lift the constraint $F(m') \leq \tau$ to the objective function, and use *quadratic penalties* to produce a smooth Lagrangian function

$$\mathcal{L}(z, \lambda) = cost(z) + \lambda P(m'), \tag{3}$$

making the primal problem

$$\begin{aligned} &\min_{z \in \mathcal{Z}^{\leq n}} \mathcal{L}(z, \lambda) \\ &\text{s.t. } m' = m + z \\ &\quad compilable(m') \wedge runnable(m') \\ &\quad m' \sim m \wedge m' \in \mathcal{M}, \end{aligned} \tag{4}$$

where λ is a positive constant, and

$$P(m') = (\max\{0, F(m') - \tau\})^2. \tag{5}$$

This treatment allows us to search for the optimal solution z^* in the whole perturbation space. For large λ it is clear that the minimum point of Problem 4 will be in a region where $P(\cdot)$ is small. Thus, for increasing λ it is expected that the corresponding points will approach the feasible region and will minimize $cost(z)$. Ideally then, as $\lambda \to \infty$ the solution point of the penalty problem will converge to a solution of the original constrained problem.

Definition 4. *(Successful evasion) Given a detector $F(x)$ and malware $m \in \mathcal{M}$, if there exists an optimal perturbation z^* to Problem 2, and the cost of z^* is controlled within a limit, i.e., $cost(z^*) \leq C_{max}$, where C_{max} is a given upper bound of the cost, then we call the detector $F(x)$ is successfully evaded or fooled by $m^* = m + z^*$, or m is successfully perturbed by z^* w.r.t. $F(x)$, denoted by the predicate $Evade_F(m, z^*)$.*

Let $S \subseteq \mathcal{M}$ be the seed corpus, we expect to screen a set of adversarial malware samples M_{adv} from S by solving Problem 2,

$$M_{adv} = \{m^* \mid m^* = m + z^* \wedge m \in S \wedge Evade_F(m, z^*)\}. \tag{6}$$

3.2. Our solution

To solve Problem 2, we propose an optimization solution strategy, called μGAN, which combines strengths of the GANs and Simulated Annealing algorithm to screen adversarial malware samples M_{adv}. The workflow of our approach consists of two phases, as illustrated in Figure 1. **Phase1** aims to craft adversarial malware samples M_{adv} by solving Problem 2 with μGAN. **Phase2** conducts *adversarial training*, i.e., retraining an enhanced malware classifier $F'(x; \theta')$ with the dataset enriched by M_{adv} to improve its robustness and accuracy.

In **Phase1**, the adversarial malware samples are generated by utilizing *program mutation* techniques. Different from the concept of *mutation* in genetic programming (Langdon and Poli, 2013), in our setting, the term *mutation* especially denotes a technique of *program transformations*—transform an Android program to a new version by introducing a *noise* or *perturbation* z. The key of this phase is to search for the *optimal* perturbation z^* that transforms malware m to the adversarial version with the *minimal edit efforts*.

Inspired by the studies E-GAN (Wang et al., 2019), MO-GAN (He et al., 2021), and MalGAN (Hu and Tan, 2022), μGAN

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

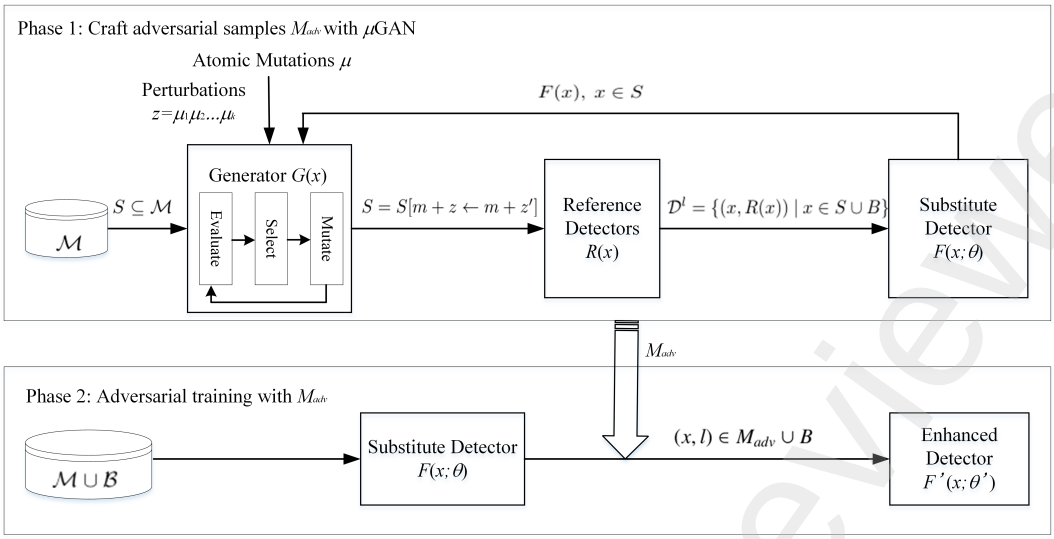

Figure 1: The schema of our approach.

combines the GANs and the Simulated Annealing algorithm to screen the optimal adversarial samples M_{adv}. μGAN includes three components, *generator* $G(m)$, *reference detectors* $R(x)$ and *substitute detector* $F(x;\theta)$. $R(x)$ is a collection of black-box third-party detectors, serving as a baseline to classify the perturbed malware m'; $F(x;\theta)$ is a classifier based on a multilayer perceptron network, which mimics the performance of $R(x)$ and outputs the probability that m' is a malicious program.

Given a population S of malware, $G(x)$ performs "Evaluate-Select-Mutate" cycle on S to calculate the fitness of each individual, select samples with higher fitness (i.e., lower values of the objective function, see Formula 3), and then mutate them to form a new population S'. Then S' is fed into $R(x)$ for labeling, forming the dataset D^l, which is subsequently used to train the substitute classifier $F(x;\theta)$. The probability produced by $F(x;\theta)$ is fed back to $G(x)$ to evaluate the fitness of each individual in S, starting the next round of "Evaluate-Select-Mutate" cycle to yield the subsequent generation of advantageous population. In summary, this cycle implements the adversarial process between the detector $F(x;\theta)$ and the generator $G(x)$—$F(x;\theta)$ aims to accurately classify the generated samples, while $G(x)$ attempts to produce samples that evade the detector. Through their mutual competition, they reach a Nash equilibrium (Nisan et al., 2007) or given number of iterations, ultimately producing an optimal set of adversarial samples.

In **Phase2**, we intend to achieve an enhanced malware classifier $F'(x;\theta')$ by augmenting the dataset with the generated adversarial malware samples M_{adv}. To this end, taking the trained substitute detector $F(x;\theta)$ as the baseline, we refine its performance by retraining with M_{adv}. Note that, instead of following traditional approaches (Idika, 2007), we develop $F(x;\theta)$ by fully exploiting strengths of the reference detectors $R(x)$, namely, training $F(x;\theta)$ to *simulate* the performance of $R(x)$. This practice reflects the well-recognized principle: "*The intelligence of the crowd surpasses that of a single sage*." As for the adversarial training, we just augment the dataset with M_{adv} and leverage traditional machine learning techniques to achieve the enhanced classifier $F'(x;\theta')$.

3.3. Training μGAN

μGAN takes malware seeds S as input and outputs M_{adv} as the result. The overall framework of μGAN is a GAN network—the generator $G(m)$ generates advantageous population, i.e., the candidate solutions of Problem 2 by employing the simulate annealing algorithm; the discriminator $F(x;\theta)$ is used to evaluate the quality of the generated solutions.

Training $F(x;\theta)$

The substitute detector $F(x;\theta)$ is a multilayer perceptron network, which *simulates* the performance of the black-box $R(x)$. We thus train $F(x;\theta)$ on the dataset

$$\mathcal{D}^l = \{(x, R(x)) \mid x \in S \cup B\},$$

where $x \in S \cup B$ is a sample (malware or benign), $R(x) \in \{0, 1\}$ is the label to x classified by $R(x)$. The loss function is as follows

$$\mathcal{L}_F = -\mathbb{E}_{x \in M^l} \log F(x;\theta) - \mathbb{E}_{x \in B^l} \log(1 - F(x;\theta)), \qquad (7)$$

where $M^l \subseteq \mathcal{D}^l$ is the set of samples recognized as malware by $R(x)$; $B^l \subseteq \mathcal{D}^l$ is the set of samples recognized as benign by $R(x)$.

Training $G(x)$

The core of $G(x)$ is a simulated annealing algorithm. It transforms a malware population S to a superior population S' by "Evaluate-Select-Mutate" cycle on S at each temperature. $G(x)$ firstly calculates the fitness of each individual; then, selects samples using **Metropolis-Hastings** sampling algorithm (Liu et al., 2017) at each temperature; finally, mutates the selected samples by introducing perturbations z. The resulted population is then used to train (via labeling by $R(x)$) $F(x;\theta)$. Subsequently the probability of $F(x;\theta)$ is fed back to $G(x)$ to start the next round of the cycle at the decreasing temperature.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Note that, the aim of μGAN is to achieve a multi-objective optimization solution—the generated samples should be both *adversarial* and *cost-minimizing*. For this purpose, the loss function for training $G(x)$ is devised as follows

$$\mathcal{L}_G = \alpha \mathcal{L}_{adv} + \beta \mathcal{L}_{obj}, \tag{8}$$

where $\mathcal{L}_{adv}$ is the loss function for generating adversarial samples; $\mathcal{L}_{obj}$ is the loss function for the objective function; α and β are of the balancing coefficients,

$$\begin{aligned} \mathcal{L}_{adv} &= -\mathbb{E}_{x \in S} \log F(G(x); \theta), \\ \mathcal{L}_{obj} &= \mathbb{E}_{x \in S} \, [cost(G(x)) + \lambda P(G(x))], \end{aligned} \tag{9}$$

here $cost(\cdot)$ and $P(\cdot)$ are of the crafting cost and penalty function for the generated samples $G(x)$, see Formula 3.

Training $F'(x; \theta)$

After obtaining adversarial malware samples M_{adv}, we further perform *adversarial training* over the substitute classifier $F(x; \theta)$ with M_{adv}. The aim of the adversarial training is to improve the malware classifier's ability to predicate newly emerging attack patterns and evolved malware variants.

The enhanced classifier $F'(x; \theta')$ retains the network architecture of $F(x; \theta)$, yet is trained on the dataset $(x, l) \in M_{adv} \cup B$, where l denotes the ground-truth label of x. The loss function is as follows

$$\mathcal{L}_{F'} = -\mathbb{E}_{x \in M_{adv}} \log F'(x; \theta') - \mathbb{E}_{x \in B} \log(1 - F'(x; \theta')). \tag{10}$$

The structured risk function of the network on $M_{adv} \cup B$ is

$$\mathcal{R}(\theta') = \mathcal{L}_{F'} + \frac{1}{2}\eta \|\theta'\|^2, \tag{11}$$

where $\|\theta'\|^2$ is the regularization term, used to prevent overfitting; η is the hyperparameter; the norm $\|\theta'\|^2$ used here is generally the Frobenius norm (Böttcher and Wenzel, 2008).

4. Implementation of our approach

4.1. Technical issues

To accomplish our approach, the following three issues must be fully addressed. The detailed solutions to these issues will be elaborated in the following subsections.

Issue 1. How to design atomic mutations? The decision variable z of Problem 4 is exactly a finite sequence of atomic mutations on malware m, so the first question to be answered is how to devise potential atomic mutations on malware. To meet the constraint set of Problem 4, we require atomic mutations μ must satisfy the following properties for any $m \in \mathcal{M}$:

$$compilable(m + \mu_i) \wedge runnable(m + \mu_i) \tag{12}$$

$$m + \mu_i \sim m \wedge m + \mu_i \in \mathcal{M} \tag{13}$$

$$m + \mu_i \mu_j = m + \mu_j \mu_i, \tag{14}$$

where properties 12 and 13 correspond to conditions **C1** and **C2** respectively, ensuring that each atomic mutation must satisfy the constraint set of Problem 4; property 14 posits the assumption that atomic mutations should be *independent* of each other, that is, sequentially applying mutations μ_i and μ_j to m in any order will result in an equivalent effect.

To tackle this issue, we devise overall 17 atomic mutations (see Section 4.2); these mutations are directly applied to the Smali codes of Apps, and do not remove APIs, permissions, or any other features of the seed malware in order to preserve the maliciousness of m intact.

Issue 2. How to evaluate the cost of perturbations $cost(z)$**?** As Problem 2 shown, we expect to find the minimal perturbation z^* for m in terms of edit efforts such that $m + z^*$ is able to successfully evade the detector F. The cost of z can be measured using the edit distance $d(m^*, m)$ between m^* and m. Since $z = \mu_1 \mu_2 \cdots \mu_k$ is a finite sequence of atomic mutations, and the atomic mutations are mutually independent (see property 14), $cost(z)$ can be evaluated as the sum of the edit efforts of each atomic mutations in z, i.e.,

$$cost(z) = d(m^*, m) = \boldsymbol{c}^\top (m^* - m) = \boldsymbol{c}^\top \vec{z} = \sum_{i=0}^{16} c_i |\mu_i|, \tag{15}$$

where $\boldsymbol{c}^\top = (c_0, c_1, \cdots, c_{16})$ is the cost vector of 17 atomic mutations, c_i is the cost of μ_i; $\vec{z} = (|\mu_0|, |\mu_1|, \cdots, |\mu_{16}|)^\top$ is the Parikh mapping (Parikh et al., 2016) of z, $|\mu_i|$ counts for the number of occurrences of μ_i in z (note that: multiple applications of an atomic mutation in a perturbation is allowable).

The cost c_i is determined by the number of changes made by μ_i to the Smali codes, which can be empirically estimated through extensive experiments. The following Section 4.3 presents our experiments designed to estimate the costs of atomic mutations, and reports the empirical results in Table 3.

Issue 3. How to search for the optimal solution of Problem 4? This is a *combinatorial optimization* problem; the objective is to minimize the function $cost(z) + \lambda P(m + z)$ over a finite, discrete solution space $\mathcal{Z}^{\leq n}$. This problem is essentially NP-hard; so we devise the scheme μGAN, which combines the GANs and the Simulated Annealing algorithm, a type of heuristic algorithms (Wu, 2021), to solve this problem. The adopt of SA allows us to balance between exploration and exploitation—exploring new areas of the search space to find potentially better solutions and exploiting known areas to refine and improve current solutions.

The primary issue when applying SA to our problem is how to construct the neighborhood structure of solutions. In our setting, solutions z take the general form of $z = \mu_1 \mu_2 \ldots \mu_k$, i.e., sequences of the atomic mutations μ_i. As the length of the optimal solution is unknown as well as the assumption is made that μ_i should be mutually independent, the neighborhood structure constructed by simply exchanging any two elements of z, like the common practice for the TSP (Traveling Salesman Problem) (Monnot and Toulouse, 2014), fails to be applicable to our problem; we thus need to design a new neighborhood structure for our solutions. In addition, we must specify a set of parameters,

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

such as temperature parameter control, cool schedule, iteration length, and acceptance probability, to govern the convergence of the algorithm and achieve superior performance and effectiveness. These points are addressed in detail in Sections 4.4 and 4.5.

The following subsections will elaborate our approach by addressing the above technical issues respectively.

4.2. Atomic mutations

We devise overall 17 atomic mutations, summarized in Table 2. The first column lists the names of atomic mutations; the second briefly describes how each atomic mutation manipulates programs; the third and forth give a simple example for each atomic mutation by showing the original and the mutated malware snippets. For space reason, some slightly more complex examples are moved to Appendix A.

These mutations are directly applied to the Smali (Freke, 2021) codes of Apps rather than Java source codes, because Smali is the native language of Android Apps—it is more directly decomposed from an APK using Apktool (Tumbleson, 2017), and offers a straightforward way to manipulate and edit instructions or opcodes, which may not be easy or possible at Java source level.

To preserve the functionality of m, we only adopt semantically equivalent program mutations, and do not remove or modify essential program features (such as APIs and permissions) of the seed malware. This treatment ensures the atomic mutations satisfy the properties 12 and 13. Furthermore, the mutations, in general, act on distinct program locations, guaranteeing mutations are mutually independent, i.e., satisfying the property 14.

As shown in Table 2, atomic mutation μ0-Nop can be viewed as the *null* mutation—no changes are made to the seed, which servers as the baseline perturbation to the seed. μ1-Repack is a basic operation for other mutations, because they require first decompiling the seed's APK file, then signing and repacking the modified files after mutations.

Atomic mutations $\mu 2 \sim \mu 10$ involve typical program refactor rules, including renaming, obfuscating, encoding, and the reflection, to manipulate seed programs; $\mu 11 \sim \mu 13$ target at the Android APIs: μ11 replaces an API in Smali codes with an API that has the same signature and the same or similar functionality; μ12 replaces an API with the corresponding deprecated version (i.e., the API no longer recommended to use); μ13 downgrades the API level used. μ12 and μ13 exploit the downgrade compatibility of Android APIs to carry out mutations, while ensuring the generated samples meet property 12.

In addition, atomic mutations $\mu 14 \sim \mu 16$ introduce perturbations by adding certain features. μ14 randomly selects an unused permission with **Normal** level and adds it to the `<uses-permission>` attribute in the `AndroidManifest.xml`. μ15 attempts to insert into the seed's code using an API snippet that does not alter the seed's functionality and is less likely to be detected as malicious segment. μ16 combines the former two—inserts an API snippet, meanwhile adds its required usage permissions into the seed's code.

4.3. Costs of perturbations

As Formula 15 shown, the cost of a perturbation z is determined by the costs c_i of the component atomic mutations μ_i and the number of occurrences of μ_i in z. The costs c_i depend on the number of changes made by μ_i to the Smali codes, which can be empirically estimated through extensive experiments.

We randomly select 50 malicious samples from the Drebin dataset (Arp et al., 2014), try to apply each atomic mutation to the samples, and count the average number of changes over the samples as the estimated values of c_i, i.e.,

$$c_i = \frac{1}{|S|} \times \sum_{m \in S} |m \to m + \mu_i|, \tag{16}$$

where S is the set of samples, $|S|$ is the cardinality of S, $|m \to m + \mu_i|$ counts the number of changes made by atomic mutations μ_i to sample $m \in S$.

The estimated values of c_i are reported in Table 3. As costs of atomic mutations are influenced by multiple factors, we thus, for concerned atomic mutations, provide various estimated values. For instance, the cost of μ4 is categorized into two levels: *private* methods and *public* methods. When applying μ4 to a private class method, say `privateMethod()`, the cost incurred is typically less than that of applying to a public class method `publicMethod()`, because the former only needs to modify the locations calling `privateMethod()` within a class, whereas the latter requires traversing the entire APK and making modifications at all locations where `publicMethod()` is called.

As the data shown in Table 3, μ0 spends zero cost because it does not make any changes to the program. The cost of μ2 is the highest, reaching 2169, because it requires modifying all locations in the APK files where the package name needs to be explicitly declared (involving the `package` and `android:name` attributes in `AndroidManifest.xml`, as well as the `.class` declarations and class path descriptions in all Smali files). In contrast, μ1 and μ14 incur the second lowest cost, only 1, because their modifications are limited to a single position.

4.4. Simulated Annealing

We use SA algorithm to implement the "Evaluate-Select-Mutate" cycle of $G(x)$. SA derives its name from analogy with an annealing process in physics and chemistry in which we start the process at high temperature and then lower the temperature slowly while maintaining thermal equilibrium. From the fundamental theory of statistical mechanics (Behrens et al., 2024), when the system is in thermal equilibrium with its surrounding environment, state i occurs with a probability defined by the following Gibbs distribution

$$p_i = \frac{1}{Z} \exp(-\frac{E_i}{k_B T}), \tag{17}$$

where E_i is the *energy* of the system when it is in state i, T is the *absolute temperature* in Kelvins, k_B is the *Boltzmann constant*, and Z is the partition function.

Utilizing SA to solve Problem 4, we interpret state i of the system as the solution z of our problem, energy E_i as the objective function (i.e., the fitness function) $f(z) = cost(z) + \lambda P(m+z)$,

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Table 2: Overall 17 atomic mutations

Atomic Mutation	Description	Original	Changed
μ0-Nop	No operation. No changes made to the APK	apk	apk
μ1-Repack	Recompile the decompiled APK into a new APK signed with custom keys.	a.apk	repack.apk
μ2-Rename Package	Reset the package name with a valid new name.	package="a.b.c"	package="r.s.t"
μ3-Rename Field Identifier	Reset a field identifier with a valid new identifier.	.field private i:I	.field private j:I
μ4-Rename Method Identifier	Reset a method identifier with a valid new identifier.	Refer to APPENDIX Appendix A.2	Refer to APPENDIX Appendix A.3
μ5-Rename Class Identifier	Reset the class identifier with a valid new identifier.	Refer to APPENDIX Appendix A.2	Refer to APPENDIX Appendix A.3
μ6-Rename Resource	Reset the resource file name with a valid new name.	Refer to APPENDIX Appendix A.4	Refer to APPENDIX Appendix A.5
μ7-Flow Obfuscation	Alter the sequence of instructions in a method by inserting deadcode, `goto`, or modifying control structures, making the logic difficult to trace.	Refer to APPENDIX Appendix A.6	Refer to APPENDIX Appendix A.7
μ8-Encode Data	Encrypt strings or arrays, protecting sensitive or personal data from being accessed by detectors statically.	Refer to APPENDIX Appendix A.8	Refer to APPENDIX Appendix A.9
μ9-Reflection	Convert a hard-coded method invocation into an implicit method call via the Java reflection mechanism.	Refer to APPENDIX Appendix A.10	Refer to APPENDIX Appendix A.11
μ10-Indirect Method Call	Convert a direct method call into an indirect method call via a deliberately devised intermediate method.	Refer to APPENDIX Appendix A.12	Refer to APPENDIX Appendix A.13
μ11-Replace APIs	Replace an API with a semantically similar API that has the same signature (i.e., the same parameter list and return value type) and functionality.	Refer to APPENDIX Appendix A.14	Refer to APPENDIX Appendix A.15
μ12-API Downgrade	Replace an API with the corresponding deprecated version.	Refer to APPENDIX Appendix A.16	Refer to APPENDIX Appendix A.17
μ13-Reduce API Level	Reduce API level in `AndroidManifest.xml` file.	Refer to APPENDIX Appendix A.18	Refer to APPENDIX Appendix A.19
μ14-Add Permission	Add Normal level permissions.	/	Refer to APPENDIX Appendix A.20
μ15-Insert API Snippets	Insert API snippets without affecting the functionality of the original program.	/	Refer to APPENDIX Appendix A.21
μ16-Add Permission & API	Add API snippets and the required usage permissions without affecting the functionality of the original program.	/	Refer to APPENDIX Appendix A.22

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Table 3: The costs of atomic mutations

Atomic Mutation	c_i
$\mu 0$-Nop	0
$\mu 1$-Repack	1
$\mu 2$-Rename Package	2169
$\mu 3$-Rename Field Identifier-private	8
$\mu 3$-Rename Field Identifier-public	34
$\mu 4$-Rename Method Identifier-private	3
$\mu 4$-Rename Method Identifier-public	7
$\mu 5$-Rename Class Identifier-private	19
$\mu 5$-Rename Class Identifier-public	78
$\mu 6$-Rename Resource	5
$\mu 7$-Flow Obfuscation	3
$\mu 8$-Encode Data	26
$\mu 9$-Reflection	17
$\mu 10$-Indirect Method Call	10
$\mu 11$-Replace API	2
$\mu 12$-API Downgrade	5
$\mu 13$-Reduce API Level	2
$\mu 14$-Add Permission	1
$\mu 15$-Insert API Snippets	2
$\mu 16$-Add Permission & API	3

and the temperature T as a control parameter. SA combines two loops—the *cooling schedule* (the outermost loop) that specifies a finite sequence of decreasing values of temperature, and *state sampling* (the innermost loop) that generates a finite number of transitions attempted at each value of the temperature, as summarized by the following algorithm framework (Caracciolo et al., 2023).

Algorithm 1: The framework of Simulated Annealing

```
Initialization: i ← i_0, k ← 0, t_0 ← t_max (initial
  temperature)
/* cooling schedule                                   */
repeat
    /* Sampling with the Metropolis-Hastings algorithm
       at each temperature t_k                        */
    for l = 0, 1, 2, ..., L_k do
        /* Mutate                                     */
        Generate a solution j from the neighborhood
          N(i) of the current solution i
        /* Evaluate                                   */
        Δf_ij ← f(j) − f(i)
        /* Select                                     */
        i ← j with the acceptance probability
          min(1, exp(−Δf_ij / t_k))
    t_{k+1} ← d(t_k), k ← k + 1
until t_k ≈ 0;
```

We use objective function $f(z) = cost(z) + \lambda P(m+z)$ to evaluate the fitness of each individual $m' = m + z \in S$, and employ the Metropolis Hastings algorithm to select next candidate solution at each temperature. The acceptance probability

$$A(i \leftarrow j) = \min(1, \exp(-\frac{\Delta f_{ij}}{t_k})) = \begin{cases} 1 & f(j) \leq f(i) \\ \exp(-\frac{\Delta f_{ij}}{t_k}) & f(j) > f(i) \end{cases} \quad (18)$$

indicates that j is definitely accepted as the current solution, if j is better than i, i.e., j leads the objective f to a lower value ($f(j) \leq f(i)$); otherwise, j is occasionally accepted only with probability $\exp(-\frac{\Delta f_{ij}}{t_k})$. This treatment allows worse solutions to be accepted and explored, thereby circumventing the pitfalls of local optima.

As aforementioned, the challenging issue here is how to construct the neighborhood structure of a solution. Considering that the length of the optimal solution is not fixed and the atomic mutations are mutually independent, we constructively define the neighborhood $N(z)$ of solution z as

$$N(z) = \{z' \mid z' = \text{1-bit-change}(z) \vee z' = z \circ \mu, \mu \in \mathcal{Z}\}, \quad (19)$$

where "1-bit-change(z)" denotes a manipulation of z by randomly substituting one atomic mutation in z; $z \circ \mu$ denotes the concatenation of z with an atomic mutation $\mu \in \mathcal{Z}$. For example, assume $z = \mu 1 \mu 2 \mu 3 \mu 4$, then $z' = \mu 1 \mu 5 \mu 3 \mu 4$ is an "1-bit-change" of z, where $\mu 2$ in z is changed to $\mu 5$ in z'; another, $z' = \mu 1 \mu 2 \mu 3 \mu 4 \mu 5$ is a neighbor of z, because $z' = z \circ \mu 5$.

This definition allows us to search for the next solution z' by changing only one atomic mutation of the current solution z, or by appending an atomic mutation to z. Clearly, this definition has taken the growth of solution length into account. For example, we always start the search process from the initial solution $z^0 = \mu 0$, the neighborhood of z^0 is

$$N(z^0) = \{\mu 1, \mu 2, \cdots, \mu 16\}.$$

We next search a solution from $N(z^0)$. Assume $\mu 1 \in N(z^0)$ is accepted as the current solution, i.e., $z^1 = \mu 1$, then we continue to search a solution from

$$N(z^1) = \{\mu 2, \mu 3, \cdots, \mu 16, \mu 1 \mu 1, \mu 1 \mu 2, \cdots, \mu 1 \mu 16\}.$$

This process continues until the optimal solution is found in finite loop steps.

4.5. Generate adversarial malware samples with μGAN

To generate adversarial malware samples M_{adv}, we must solve Problem 4 to determine the optimal perturbation z^* for each malware $m \in S$ such that the predicate $Evade_F(m, z^*)$ holds, as Formula 6 shown. The following Algorithm 2 presents how to generate M_{adv} with μGAN framework, which takes seeds $S \subseteq \mathcal{M}$ as input, M_{adv} as output.

For the penalty Problem 4, ideally, when constant $\lambda \to \infty$ the solution of the problem will converge to a solution of the original constrained Problem 2. However, in practice, we always give λ a specific large value when solving the problem with a

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Algorithm 2: Generate M_{adv} with μGAN

Input: $S \subseteq \mathcal{M}$, $m_i \in S$, $B \subseteq \mathcal{B}$
Output: M_{adv}

1 /* fitness function: $f(z) = cost(z) + \lambda P(m + z)$ */
2 Initialization: $z_i \leftarrow z^0$, $z_i^* \leftarrow z_i$, $k \leftarrow 0$, $t_0 \leftarrow t_{max}$
3 $M_{adv} \leftarrow \emptyset$
4 **repeat**
5 /* Training $F(x;\theta)$ */
6 Labeling $x \in S \cup B$ with $R(x)$ to form the dataset $D^l = \{(x, R(x)) \mid x \in S \cup B\}$
7 Training $F(x;\theta)$ with D^l
8 **foreach** $m_i + z_i \in S$ **do**
9 /* Mutate */
10 select $z_i' \in N(z_i)$ at random
11 /* Evaluate */
12 calculate $f(z_i')$, $f(z_i)$
13 /* Select */
14 **if** $\texttt{rand}(0, 1) < A(z_i \leftarrow z_i')$ **then**
15 $z_i \leftarrow z_i'$
16 /* Update $m_i + z_i$ with $m_i + z_i'$ */
17 $S = S[m_i + z_i \leftarrow m_i + z_i']$
18 **if** $f(z_i) < f(z_i^*)$ **then**
19 $z_i^* \leftarrow z_i$
20 $t_{k+1} \leftarrow \alpha t_k$, $k \leftarrow k + 1$
21 **until** $t_k \leq \epsilon$;
22 **foreach** $m_i \in S$ **do**
23 **if** $F(m_i + z_i^*) \leq \tau \wedge cost(z_i^*) \leq C_{max}$ **then**
24 $M_{adv} \leftarrow M_{adv} \cup \{m_i + z_i^*\}$

computer, thereby the solution of Problem 4 may violate the constraint $F(m^*) \leq \tau$ of Problem 2. More specifically, there are two cases for the optimal solution z^* of Problem 4:

1. $\tau < F(m + z^*) \leq F(m)$, i.e., applying z^* to m only reduces the likelihood that $m^* = m + z^*$ is detected as malicious, but m^* remains detectable. This case indicates that no feasible solution is available for Problem 2.
2. $F(m + z^*) \leq \tau$, i.e., m^* successfully evades detection, indicating that z^* is the solution of Problem 2. In this case, m^* is added to M_{adv} as an adversarial malware sample.

In Algorithm 2, $f(z)$ is the objective function of Problem 4, which corresponds to the *energy* E of the Simulated Annealing. For each malware $m_i \in S$, we employ μGAN to find the optimal perturbation z_i^* for m_i; if $m_i + z_i^*$ successfully bypasses detector F, and the cost of z_i^* is controlled within a limit C_{max}, then $m_i^* = m_i + z_i^*$ is considered as an adversarial sample, and is added to M_{adv} (at Lines 23-24).

The μGAN algorithm ranges over Lines 4-21. In each cooling loop, we first train the substitute detector $F(x;\theta)$ with dataset D^l (at Lines 6-7), which will be used to evaluate the fitness of each individual in S (at Line 12). For each sample $m_i + z_i \in S$, it is required to undergo three stages of mutation, evaluation, and selection (at Lines 8-19). The optimized samples obtained will replace old ones to form new individuals of S (at Line 17). The temperature decrement function $d(t_k)$ is instantiated as a linear function αt_k (at Line 20), where typical values of α lie between 0.8 and 0.99; when t_k is less than a small positive number ϵ, it is considered that the temperature is close to 0 degrees, and the SA algorithm stops (at Line 21).

4.6. Adversarial training

In **Phase2**, we intend to achieve an enhanced malware classifier $F'(x;\theta')$ by augmenting the dataset with the generated adversarial malware samples M_{adv}. The goal of $F'(x;\theta')$ is to improve the classifier's robustness against the adversarial samples. The training process is accomplished by minimizing the empirical risk function (Formula 11), indicating that $F'(x;\theta')$ is expected to retain the power of $F(x;\theta)$ as well as be able to classify the adversarial samples correctly. The training process is presented as the following Algorithm 3.

Algorithm 3: Adversarial training with M_{adv}

Input: $F(x;\theta)$, M_{adv}, $B \subseteq \mathcal{B}$
Output: $F'(x;\theta')$

1 Initialization: $\theta' \leftarrow \theta$
2 **for** $i \leftarrow 1$ to N **do**
3 **foreach** $(x, l) \in M_{adv} \cup B$ **do**
4 $g \leftarrow \nabla_{\theta'} \mathcal{L}_{F'}$
5 $\theta' \leftarrow Adam(\theta', \lambda, g)$
6 **return** $F'(x;\theta')$

Algorithm 3 takes the substitute detector $F(x;\theta)$ and the generated adversarial samples M_{adv} as inputs. The weights of $F'(x;\theta')$ are initialized with those of $F(x;\theta)$, meaning that $F'(x;\theta')$ retains the performance of $F(x;\theta)$ in the beginning. The training process is implemented at Lines 2-5 on the dataset $M_{adv} \cup B$. Firstly, the gradient $\nabla_{\theta'} \mathcal{L}_{F'}$ is computed using the backpropagation algorithm, then the weight matrix θ' is updated by using the Adam optimization algorithm

$$\theta' = \theta' - \lambda \nabla_{\theta'} \mathcal{L}_{F'},$$

where λ denotes the learning rate set to 0.01. The training process is iterated until the loss function converges or the maximum number of iterations is reached.

5. Evaluation

In this section, we address the following five research questions (**RQs**) through experimental evaluation:

- **RQ1**. To accomplish our approach, is the process "*decompiling → applying mutations → repackaging*" is feasible?
- **RQ2**. Can mutations help malware samples escape detection?
- **RQ3**. Can the generated adversarial malware samples successfully bypass detection mechanisms? Is the evasion capability correlated with their family affiliations?

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

- **RQ4**. What is the performance of training $F(x;\theta)$ and $F'(x;\theta')$ using our approach?
- **RQ5**. Compared to the state-of-the-art Android malware detectors, what is the performance of our classifier?

Hardware and Software Environment

The operating system used in our experiment is Windows 11 64 bit, with 16GB of memory and 11th Gen Intel(R) Core(TM) i5-1155G7 @ 2.50GHz. The emulator is Android 7.1 Pixel NDE63V. We use Apktool to decompile APK files, and use Python 3.8 and Tensorflow 2.9.0 to train the neural network models. In addition, we use 70 malware detectors integrated in VirusTotal v3 as the reference detectors.

Evaluation Metrics and Datasets

To address these **RQs**, we adopt these metrics shown in Table 4 to comprehensively evaluate the performance of our approach. A variety of datasets are publicly available for Android malware detection research, which typically contain a large number of malware and benign samples used to train and test malware detectors. Here we choose malware samples from three datasets: Drebin (Arp et al., 2014), Maldroid (Mahdavifar et al., 2020), and VirusShare, which cover a wide range of Apps types and malware families; the benign samples are selected from the Google Play Store, and have been subjected to stringent checking to ensure their benign status. The statistics of these datasets is shown in Table 5.

5.1. Answer to RQ1

To implement our approach, we need to achieve such a process: decompile an APK, apply mutations to the decompiled Smali codes, then sign and repackage the mutated codes. However, this process may fail in certain cases, because some protection mechanisms (e.g., code hardening techniques, code integrity verification mechanisms, and signature mechanisms) can thwart successful decompilation and repackaging of APKs. Therefore, we must firstly examine *whether this process is practical for the majority of malware samples*. To this end, we have collected samples from 20 major malware families, and subjected each sample to the aforementioned process; then counted the samples that successfully passed through this process. The results of our experiment are presented in Table 6.

As shown in Table 6, the majority of malware families (14 out of 20 families) can completely pass through the process of "decompiling-mutating-repackaging", and the average pass rate is 87%. However, some samples from certain families fail to be decompiled, repackaged, or installed, caused by the protection of defense mechanisms. For example, **BaseBridge** employs commercial APK protection tools (such as DexGuard) to encrypt and protect APK files, which increases the difficulty of decompilation, causing its SR is only 24%. **GingerMaster** adopts dynamic loading techniques to conceal malicious behaviors, making it impossible to repackage the mutated APK, resulting in an SR of 75%. **DroidKungFu**, by means of hard coding and detecting the presence of emulators, possesses powerful anti-debugging and anti-emulator capabilities, which effectively prevent Apps from being installed or running in a virtual environment, leading to an SR of 45%.

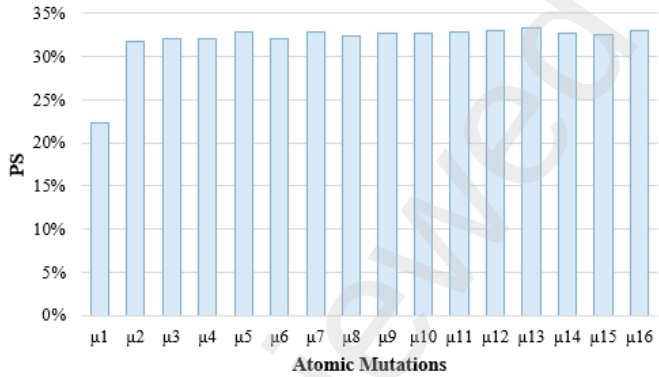

Figure 2: The average PS value of each atomic mutation.

5.2. Answer to RQ2

To evaluate how atomic mutations help malware samples evade detection, we introduce a metric PS (*Perturbation Sensitivity*) to measure the ability of an atomic mutation μ_i to deceive detectors

$$PS(\mu_i) = \frac{F(m) - F(m')}{F(m)}, \tag{20}$$

where $m' = m + \mu_i$, $m \in S$, $F(m)$ is the substitute detector, which outputs the likelihood that m is identified as malware. The higher the $PS(\mu_i)$ value, the stronger the ability of μ_i to aid m in evading detection.

We apply the atomic mutations shown in Table 2 to 50 malware samples, then calculate the average PS value for each atomic mutation, as shown in Figure 2. Among them, $\mu 13$ has the highest PS value of 33.33%, indicating that introducing $\mu 13$ is more likely to evade detection mechanisms. This is because most detectors use API features as the criteria for classification; reducing API level enables malware to bypass these detectors. In contrast, $\mu 1$ exhibits the lowest PS value of 22.32%, indicating it is challenging to deceive detectors by simply repackaging seed samples. In fact, $\mu 1$ only alters the signature and the structure of the seed, which can be easily detected by various anti-tampering mechanisms, such as signature verification and file integrity checking tools.

5.3. Answer to RQ3

To investigate how effective the generated adversarial malware samples bypass detection, we selected 40 malware samples from different families as seeds, and crafted adversarial samples using our approach. These samples are detected by the substitute detector $F(\cdot)$; the detection results are reported in Figure 3.

As shown in this figure, the majority of the generated adversarial samples exhibit a reduced detection rate of malware compared to that of the seed, and can successfully evade detection at a threshold of $\tau = 0.5$. However, we found that, from the App "Wonders of the world Live Wallpaper" we could not derive a valid adversarial sample, because all the perturbed versions could not be installed in the simulator. Even worse, after adding

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Table 4: Evaluation metrics

Metrics	Description	Formula for calculation
TP	Number of samples correctly classified as malicious	/
TN	Number of samples correctly classified as benign	/
FP	Number of samples misclassified as malicious	/
FN	Number of samples misclassified as benign	/
$PS(\mu_i)$	Perturbation Sensitivity of atomic mutations μ_i, i.e., the ability of μ_i to deceive classifiers. $m' = m + \mu_i$	$\frac{F(m)-F(m')}{F(m)}$
SR	Success Rate of generating adversarial samples. N is the total number of seeds, n is the number of the adversarial samples successfully generated from N seeds.	n/N
Accuracy	Proportion of correctly classified as malicious and benign samples	$\frac{TP+TN}{TP+TN+FP+FN}$
Recall	Proportion of all actual malicious samples correctly identified as malicious	$TP/(TP+TN)$
FPR	Proportion of samples that are actually benign but misclassified as malicious	$FP/(TN+FP)$

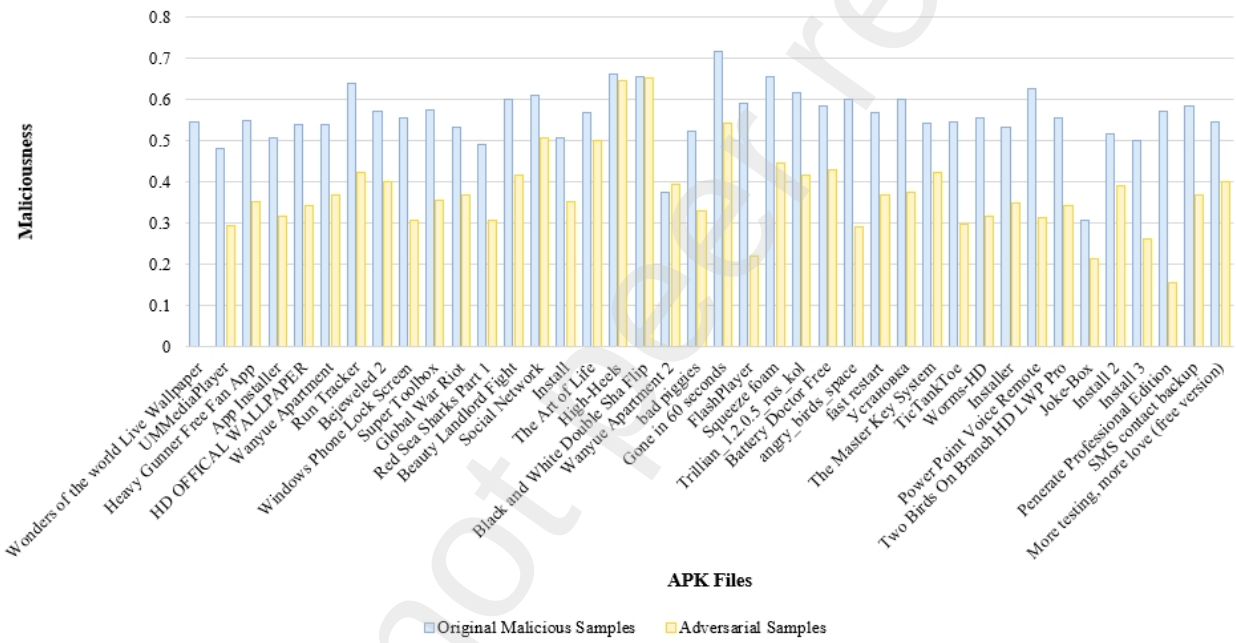

Figure 3: The evasion capability of adversarial malware samples.

Table 5: The datasets

Category	Datasets	# samples
	Drebin	1358
Malware samples	Maldroid	1160
	VirusShare	600
Benign samples	Google Play Store	2312

perturbations, the App "Wanyue Apartment 2" exhibited an increased detection rate, indicating that the desired adversarial samples could not be achieved as constraint **C3** was breached.

Furthermore, in order to examine whether the evasion capabilities of adversarial samples are correlated to their families, we calculate, for each family, the average detection rate of the seeds and that of the corresponding adversarial samples

$$F(family)_{seed} = \frac{1}{|family|} \sum_{m \in family} F(m) \tag{21}$$

$$F(family)_{adv} = \frac{1}{|family|} \sum_{m \in family} F(m^*), \tag{22}$$

where m^* is the adversarial malware sample of $m \in family$; $|family|$ counts the number of samples in $family$. The results are presented in Figure 4, in which, for each family, $F(family)_{adv} < F(family)_{seed}$, indicating that our approach can generally enhance the evasion capability for each family. Among them, the family **DroidKungFu** exhibits the smallest reduction of 1.76%, suggesting that the adversarial samples from **DroidKungFu** have the weakest evasion capabilities. On the contrary, **Fatakr** and **Boxer** families have the greatest reduction values of 37.21% and 41.51% respectively, showing that the adversarial samples

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Table 6: The SR of 20 major malware families

Malware family	# samples	# samples passed successfully	SR	Causes
FakeInstaller	100	100	100%	
SMSreg	90	90	100%	
Plankton	80	60	75%	Failed to be installed
Opfake	80	64	80%	Failed to be repackaged
FakeDoc	70	70	100%	
GingerMaster	60	45	75%	Failed to be repackaged
BaseBridge	50	12	24%	Failed to be decompiled
DroidDream	50	50	100%	
Adrd	40	40	100%	
Iconosys	40	40	100%	
DroidKungFu	40	18	45%	Failed to be installed
Kmin	30	30	100%	
SendPay	30	30	100%	
Boxer	30	30	100%	
Goldream	20	20	100%	
FakeRun	20	20	100%	
Penetho	20	20	100%	
Nyleaker	20	20	100%	
Fatakr	10	10	100%	
ExploitLinuxLotoor	10	6	60%	Failed to be installed

#samples: the total number of samples in each family
#samples passed successfully: the number of samples that successfully passed through the process
$SR = \frac{\text{\#samples passed successfully}}{\text{\#samples}}$: the pass rate of samples in each family
Causes: the potential causes for pass failure

generated from these families are more likely to evade detection.

Additionally, we compared μGAN with two existing representative studies, i.e., Mystique (Meng et al., 2016) and MalGAN (Hu and Tan, 2022) to evaluate the performance of generating adversarial malware samples. To this end, we selected 50 malware samples as seeds, and generated adversarial samples using Mystique, MalGAN, and μGAN, respectively. The performance is measured by two metrics: the average detection rate (ADR) of malware and the number of iterations (#iterations) generating the adversarial samples.

The result of comparison is illustrated in Table 7. It shows that, μGAN reaches a detection rate of 10.0% in only 5 iterations, whereas Mystique and MalGAN, even after 50 iterations, still show detection rates above 10.0%. This indicates that μGAN can generate adversarial samples with a faster convergence than Mystique and MalGAN. Moreover, once iterations have stabilized (after 50 iterations for Mystique and MalGAN, and 5 iterations for μGAN), the average detection rate achieved by μGAN is notably lower than that of Mystique and MalGAN. The superiority of μGAN arises from the fact that Mystique relies on the constructed Android malware meta-model to synthesize adversarial samples, which requires more iterations to achieve the desired combination of features. In contrast, μGAN integrates strengths of program mutation techniques and the intelligent optimization algorithm to craft adversarial samples, thereby achieving a lower *ADR* with fewer iterations.

Table 7: Compare μGAN with Mystique and MalGAN

Mystique		MalGAN		μGAN	
#iter	*ADR*(%)	#iter	*ADR*(%)	#iter	*ADR*(%)
10	30.7	10	16.3	1	15.9
20	16.8	20	13.4	2	12.5
30	16.4	30	12.6	3	11.7
40	15.8	40	11.6	4	11.1
50	13.6	50	10.7	5	10.0

5.4. Answer to RQ4

Inspired by Drebin (Arp et al., 2014), we select multiple features (e.g., hardware components, permissions, App components, Intent Filters, APIs, and network addresses) (see Table 8) to form feature vectors for Apps. Following the method aforementioned in Section 3.3, we trained the substitute classifier $F(\cdot)$ and the enhanced detector $F'(x;\theta')$. The performances of training $F(\cdot)$ and $F'(\cdot)$ are reported in the following Figures 5, 6, and Table 9 respectively.

Table 8: The collected features

Sources	Features	#Features
AndroidManifest.xml	Hardware Components	72
	Permissions	3812
	App Components	218951
Bytecode	API Calls	1048
	Network Addresses	310447

We extract 40% and 20% of the samples from the dataset as

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

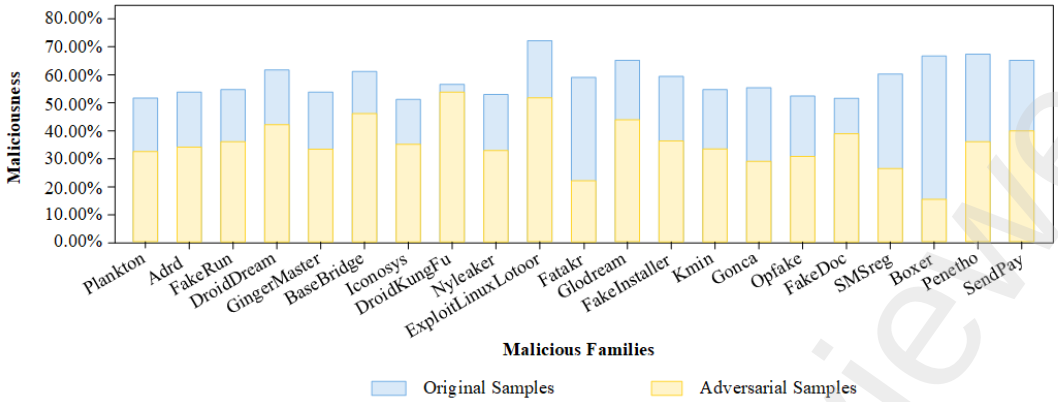

Figure 4: The evasion capability of some malware families.

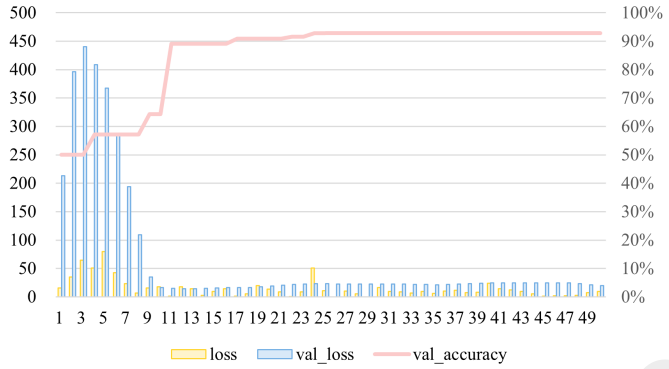

Figure 5: The performance of training $F(\cdot)$.

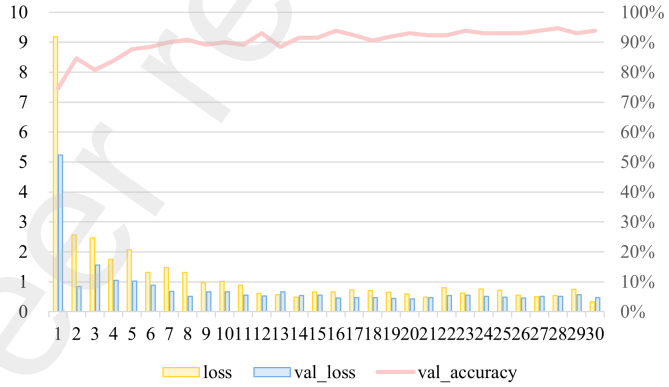

Figure 6: The performance of training $F'(\cdot)$.

the training set and the validation set for $F(\cdot)$, respectively. The loss (*loss*), validation loss (*val_loss*), and accuracy (*val_accuracy*) obtained during the training process of $F(\cdot)$ are shown in Figure 5. In the initial rounds of training, the loss value (either *loss* or *val_loss*) is relatively high; as training progresses, the loss value gradually decreases while the *val_accuracy* steadily reaches 92.86% after 50 epochs.

Next, we selected 200 malware samples from the dataset as the seeds, and generated 173 adversarial malware samples (note: 27 samples are discarded, as the adversarial samples cannot be derived from them in the given time and cost budgets). We re-trained $F(\cdot)$ with the generated adversarial malware samples and achieved the enhanced malware classifier $F'(\cdot)$. The performance of training $F'(\cdot)$ is shown in Figure 6. After 30 epochs, the accuracy of $F'(\cdot)$ reaches 94.16%. Table 9 summarizes the *Accuracy*, *Recall*, and *FPR* of $F(\cdot)$ and $F'(\cdot)$ respectively, showing that $F'(\cdot)$ outperforms $F(\cdot)$ in terms of accuracy and recall, while exhibits a lower *FPR*.

Table 9: The evaluation results of $F(\cdot)$ and $F'(\cdot)$

Detectors	*Accuracy*	*Recall*	*FPR*
$F(\cdot)$	0.93	0.97	0.12
$F'(\cdot)$	0.94	0.99	0.02

5.5. Answer to RQ5

We selected four state-of-the-art malware detectors, namely *Avast*, *AVG*, *Kaspersky*, and *Microsoft*, and conducted a comparative experiment with our enhanced classifier $F'(\cdot)$ over three key metrics: *Accuracy*, *Recall*, and *FPR*. As the result shown in Table 10, $F'(\cdot)$ outperforms the other four detectors in terms of *Recall*, indicating that $F'(\cdot)$ performs best in identifying true positives. Meanwhile, $F'(\cdot)$ improves metrics of *Accuracy* and *FPR* significantly compared to *Avast*, *AVG*, and *Microsoft*, suggesting that $F'(\cdot)$ achieves high performance in correctly detecting both malware and benign samples.

Table 10: Compare $F'(\cdot)$ with state-of-the-art classifiers

	Avast	**AVG**	**Kaspersky**	**Microsoft**	$F'(\cdot)$
Accuracy	0.89	0.88	0.97	0.79	0.94
Recall	0.58	0.56	0.93	0.38	0.99
FPR	0.03	0.04	0.02	0.03	0.02

Note that, Kaspersky slightly surpasses our detector $F'(\cdot)$ in terms of *Accuracy*, as it leverages a powerful similarity detection algorithm, known as *SmartHash* (Samanta and Jain, 2024), to detect against new and polymorphic malware. But it retains an *FPR* on par with $F'(\cdot)$ and underperforms in terms of *Recall*, indicating Kaspersky's limits in identifying all actual malware

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

samples. To sum up, $F'(\cdot)$ outperforms the other detectors in nearly overall performance metrics.

5.6. The findings

Regarding **RQ1**, we find that samples from 14 distinct malware families are able to successfully pass the process of decompiling, mutating, and repackaging with an average success rate of 87%, proving that our approach is feasible for the majority of malware samples. For **RQ2**, we conclude that introducing atomic mutations facilitates malware samples to evade security detection; different mutations typically exhibit varying evasion capabilities (measured by *PS* values). Notably, $\mu 9$ and $\mu 13$ prove more favorable for malware evasion with lower editing costs, whereas $\mu 1$ is less effective in evading detection mechanisms. For **RQ3**, it is discovered that 95% of the generated adversarial samples can successfully evade detection at a threshold $\tau = 0.5$. μGAN can generate adversarial samples with a faster convergence and lower detection rate than Mystique and MalGAN. As for **RQ4**, the enhanced classifier $F'(\cdot)$ obtained through adversarial training achieves an *Accuracy* of 0.94, showing its strong robustness in detecting against newly emerging and evolving malware variants. Finally, in **RQ5**, compared with the leading malware detectors, *Avast*, *AVG*, *Kaspersky*, and *Microsoft*, the enhanced classifier $F'(\cdot)$ exhibits a superior detection performance in nearly overall metrics.

6. Related work

6.1. Machine Learning-based detection methods

Detecting malware with ML techniques mainly focus on selecting and extracting proper features from program codes or running traces. WaffleDetector (Sun et al., 2017) and RanDroid (Koli, 2018) extract sensitive permissions and API calls as the features; TFDroid (Lou et al., 2019) analyzes features of sensitive data flows extracted by FlowDroid and implements detection using SVM algorithm; Mat et al. (2022) extracts permissions and employs a Naive Bayes classifier to achieve a high accuracy malware detector; Ceschin et al. (2023) focuses on the concept drift of dataset and uses an adaptive random forest as a dynamically updated detector; DATDroid (Thangaveloo et al., 2020) extracts runtime features, including system calls, CPU and memory usages, and network packets, to detect malware; MEGDroid (Hasan et al., 2021) extracts all possible event sources and uses dynamic analysis to improve the event generation process in Android malware detection.

Recently, deep learning methods are also widely used in the field of Android malware detection. For example, MAPAS (Kim et al., 2022) employs Convolutional Neural Networks (CNNs) to analyze API call graphs of malware, extract high-weight features, and construct a lightweight classifier with the Jaccard similarity algorithm. DroidRL (Wu et al., 2023) uses Double Deep Q-Networks (DDQNs) to explore the optimal subset of features during training, and also employs Recurrent Neural Networks and their variants (such as LSTM and GRU) as the decision network for DDQNs to handle sequential feature selection tasks. HAWK (Hei et al., 2024) constructs a heterogeneous information network and utilizes Graph Neural Networks (GNNs) to uncover complex semantic relationships between Android applications, achieving an efficient malware classifier. Qaisar and Li (2022) proposed a multimodal information fusion approach, which combines static, dynamic, and visual features together and employs lazy learning techniques to achieve efficient malware prediction and classification.

6.2. Adversarial samples generation

The concept of *adversarial perturbations* was first introduced by Szegedy et al. (2013) in 2013, which are small, carefully crafted changes made to input data with the goal of causing machine learning models, especially deep neural networks, to produce incorrect outputs or classifications. Papernot et al. (2016) explored the vulnerabilities of deep learning models, particularly in the context of adversarial attacks, and discussed various algorithms, such as FGSM (Fast Gradient Sign Method), for generating adversarial examples. Focusing on generating adversarial samples for Android malware, Grosse et al. (2017) extended the method proposed by Papernot et al. (2016) via adding perturbations to the `AndroidManifest.xml` file that only require adding a single line of code. Hu and Tan (2022) presented a novel approach, referred to as MalGAN, to generate adversarial malware examples that can bypass black-box detection systems.

Alternatively, Mystique (Meng et al., 2016) proposed a metamodel to capture common attack and evasion features in malware, and developed a framework to automatically generate Android malware using software product line engineering methods. To audit Anti-Malware Tools (AMT), following the framework of Mystique, researchers further implemented a service-oriented malware generation system named Mystique-S (Xue et al., 2017). This system can automatically select attack features and deliver corresponding malicious payloads at runtime.

To Evaluate the effectiveness of commercial Android anti-malware products, Rastogi et al. (2014) developed DroidChameleon, a systematic framework that employs various program transformation techniques to automatically modify Android applications and simulates how malware might evade detection. This study revealed that all evaluated anti-malware products were vulnerable to common evasion techniques, and in many cases, could be easily defeated by applying minor transformations to known malware. Additionally, Maiorca et al. (2015) proposed multiple strategies to obfuscate malicious software, and highlighted that simple obfuscation transformations could effectively deceive anti-malware engines. Aydogan and Sen (2015) proposed an approach to evolving mobile malware variants using genetic programming. AdvDroidZero (He et al., 2023) introduced an efficient query-based attack framework that requires only a small number of queries to generate adversarial Android malware under a zero knowledge setting, with approximately 15 queries sufficient to generate 80% of such malware within a few minutes.

6.3. Adversarial training

In order to reduce the misclassification rate of classifiers, *defensive distillation* or *adversarial training* was proposed to en-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

hance the classifier (Grosse et al., 2017). The former is achieved by modifying the normalization function of the softmax layer, whereas the latter uses generated adversarial malware samples to retrain ML-based detectors. Both methods can enhance the robustness of detectors, but defense distillation still exhibits a relatively high misclassification rate, we thus opt for the adversarial training method to promote the performance of detectors, as presented in this paper.

In fact, adversarial training was originally proposed by Szegedy et al. (2013) for robust image recognition and later developed by Grosse et al. (2017) for malware detection. Its core objective is to improve the classifiers' generalization, namely, the prediction of samples outside the training set. Good generalization makes a classifier less sensitive to small perturbations, thereby increasing its resilience to adversarial samples (Li et al., 2021). Adversarial training can be applied to the malware classification task (Wang et al., 2021) to enhance the classifier's ability to predict emerging attack patterns and evolving malware variants. GMADV (Li et al., 2024a) mixes the generated adversarial images with the original RGB Markov images to form an augmented dataset, then performs adversarial training by feeding this dataset into the VGG13 model to improve the robustness of the classifier. MaMaDroid2.0 (Berger et al., 2022) incorporates permission features into the CFG (Control Flow Graph) features used in MaMaDroid1.0, and retrains the decision tree model, leading to a significant improvement in detection rate when facing structural attacks.

7. Conclusion

In this paper, we propose a mutation-based cost optimal adversarial malware generation approach against continuously evolving Android malware variants. This approach applies program transformations (i.e., atomic mutations) to perturb seed samples in order to enable them to evade detection systems successfully. We devise an optimization strategy μGAN, which combines the GANs and the Simulated Annealing algorithm, to screen optimal adversarial samples from seeds. Moreover, we retrain an enhanced malware detector by augmenting the dataset with the generated adversarial samples to improve the resilience of detection.

The key challenge of our approach is how to solve the combinatorial optimization problem 2. Here, we leverage the framework of the GANs and the Simulated Annealing algorithm to accomplish an implementation solution. In fact, reinforcement learning, as a commonly used optimization strategy, has been widely applied to solving combinatorial optimization problems. Therefore, in the near future work, we will focus on implementing and evaluating reinforcement learning algorithms to enhance our current method and compare their performance with existing solutions.

Acknowledgment

This work is supported by the National Natural Science Foundation of China under Grant No.12071367.

References

360-Mobile-Security, 2024. 2023 mobile security report. Available at https://pop.shouji.360.cn/safe_report/Mobile-Security-Report-202312.pdf .

Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K., 2014. Drebin: Effective and explainable detection of android malware in your pocket, in: Network and Distributed System Security Symposium, pp. 23–26. doi:https://doi.org/10.14722/ndss.2014.23247.

Arya, S., Peddoju, S.K., 2024. (poster) federated learning assisted model for android malware detection using gannet optimization algorithm, in: 2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT), pp. 777–779. doi:https://doi.org/10.1109/DCOSS-IoT61029.2024.00122.

Arzt, S., Rasthofer, S., Fritz, C., Bodden, E., Bartel, A., Klein, J., Le Traon, Y., Octeau, D., McDaniel, P., 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. Acm Sigplan Notices 49, 259–269. doi:https://doi.org/10.1145/2594291.2594299.

Aydogan, E., Sen, S., 2015. Automatic generation of mobile malwares using genetic programming, in: Applications of Evolutionary Computation, Springer International Publishing. pp. 745–756. doi:https://doi.org/10.1007/978-3-319-16549-3_60.

Behrens, F., Mainali, N., Marullo, C., Lee, S., Sorscher, B., Sompolinsky, H., 2024. Statistical mechanics of deep learning. Journal of Statistical Mechanics: Theory and Experiment 2024, 104007. doi:https://doi.org/10.1088/1742-5468/ad3a60.

Berger, H., Hajaj, C., Mariconti, E., Dvir, A., 2022. Mamadroid2. 0–the holes of control flow graphs. arXiv doi:https://doi.org/10.48550/arXiv.2202.13922.

Böttcher, A., Wenzel, D., 2008. The frobenius norm and the commutator. Linear Algebra and its Applications 429, 1864–1885. doi:https://doi.org/10.1016/j.laa.2008.05.020.

Cani, A., Gaudesi, M., Sanchez, E., Squillero, G., Tonda, A., 2014. Towards automated malware creation: Code generation and code integration, in: Proceedings of the 29th Annual ACM Symposium on Applied Computing, Association for Computing Machinery. p. 157–160. doi:https://doi.org/10.1145/2554850.2555157.

Caracciolo, S., Hartmann, A., Kirkpatrick, S., Weigel, M., 2023. Simulated annealing, optimization, searching for ground states. chapter Chapter 1. pp. 1–20. doi:https://doi.org/10.1142/9789811273926_0001.

Catak, F.O., Ahmed, J., Sahinbas, K., Khand, Z.H., 2021. Data augmentation based malware detection using convolutional neural networks. PeerJ. Computer science 7, e346. doi:https://doi.org/10.7717/PEERJ-CS.346.

Ceschin, F., Botacin, M., Gomes, H.M., Pinagé, F., Oliveira, L.S., Grégio, A., 2023. Fast & furious: On the modelling of malware detection as an evolving data stream. Expert Syst. Appl. 212, 0957–4174. doi:https://doi.org/10.1016/j.eswa.2022.118590.

Chen, K., Wang, P., Lee, Y., Wang, X., Zhang, N., Huang, H., Zou, W., Liu, P., 2015. Finding unknown malice in 10 seconds: mass vetting for new threats at the google-play scale, in: Proceedings of the 24th USENIX Conference on Security Symposium, USENIX Association. p. 659–674. doi:https://doi.org/10.5555/2831143.2831185.

Chen, L., Sultana, S., Sahita, R., 2018. Henet: A deep learning approach on intel® processor trace for effective exploit detection, in: 2018 IEEE Security and Privacy Workshops (SPW), pp. 109–115. doi:https://doi.org/10.1109/SPW.2018.00025.

Conti, M., P., V., Vitella, A., 2022. Obfuscation detection in android applications using deep learning. Journal of Information Security and Applications 70, 103311. doi:https://doi.org/10.1016/j.jisa.2022.103311.

D., A., K.A., V.K., S., S.C., P., V., 2019. Malware traffic classification using principal component analysis and artificial neural network for extreme surveillance. Computer Communications 147, 50–57. doi:https://doi.org/10.1016/j.comcom.2019.08.003.

Demetrio, L., Biggio, B., Lagorio, G., Roli, F., Armando, A., 2021. Functionality-preserving black-box optimization of adversarial windows malware. IEEE Transactions on Information Forensics and Security 16, 3469–3478. doi:https://doi.org/10.1109/TIFS.2021.3082330.

Dolev, S., Ghanayim, M., Binun, A., Frenkel, S., Sun, Y.S., 2017. Relationship of jaccard and edit distance in malware clustering and online identification, in: 2017 IEEE 16th International Symposium on Network Computing and Ap-

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

plications (NCA), pp. 1–5. doi:https://doi.org/10.1109/NCA.2017.8171380.

Dong, Z., Hu, Q., Zhang, Z., Zhao, J., 2024. On the effectiveness of graph data augmentation for source code learning. Knowledge-Based Systems 285, 111328. doi:https://doi.org/10.1016/j.knosys.2023.111328.

Faruki, P., Bhan, R., Jain, V., Bhatia, S., El Madhoun, N., Pamula, R., 2023. A survey and evaluation of android-based malware evasion techniques and detection frameworks. Information 14. doi:https://doi.org/10.3390/info14070374.

Fletcher, R., 1975. An ideal penalty function for constrained optimization. IMA Journal of Applied Mathematics 15, 319–342. doi:https://doi.org/10.1093/imamat/15.3.319.

Freke, J., 2021. smali downloads. https://bitbucket.org/JesusFreke/smali/downloads/. Accessed: Sep. 19, 2024.

Goodfellow, I., Bengio, Y., Courville, A., 2016. Deep Learning. MIT Press. http://www.deeplearningbook.org.

Grosse, K., Papernot, N., Manoharan, P., Backes, M., Mcdaniel, P., 2016. Adversarial perturbations against deep neural networks for malware classification. ArXiv doi:https://doi.org/10.48550/arXiv.1606.04435.

Grosse, K., Papernot, N., Manoharan, P., Backes, M., McDaniel, P., 2017. Adversarial examples for malware detection, in: Computer Security – ESORICS 2017, pp. 62–79. doi:https://doi.org/10.1007/978-3-319-66399-9_4.

Gupta, R., Pal, S., Kanade, A., Shevade, S., 2017. Deepfix: fixing common c language errors by deep learning, in: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, AAAI Press. p. 1345–1351. doi:https://doi.org/10.1609/aaai.v31i1.10742.

Han, X., Yu, X., Pasquier, T.F.J., Li, D., Rhee, J., Mickens, J., Seltzer, M.I., Chen, H., 2021. Sigl: Securing software installations through deep graph learning, in: 30th USENIX Security Symposium (USENIX Security 21), USENIX Association. pp. 2345–2362. doi:https://doi.org/10.48550/arXiv.2008.11533.

Hasan, H., Tork Ladani, B., Zamani, B., 2021. Megdroid: A model-driven event generation framework for dynamic android malware analysis. Information and Software Technology 135, 106569. doi:https://doi.org/10.1016/j.infsof.2021.106569.

He, C., Huang, S., Cheng, R., Tan, K.C., Jin, Y., 2021. Evolutionary multiobjective optimization driven by generative adversarial networks (gans). IEEE Transactions on Cybernetics 51, 3129–3142. doi:https://doi.org/10.1109/TCYB.2020.2985081.

He, P., Xia, Y., Zhang, X., Ji, S., 2023. Efficient query-based attack against ml-based android malware detection under zero knowledge setting, in: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, Association for Computing Machinery. p. 90–104. doi:https://doi.org/10.1145/3576915.3623117.

Hei, Y., Yang, R., Peng, H., Wang, L., Xu, X., Liu, J., Liu, H., Xu, J., Sun, L., 2024. Hawk: Rapid android malware detection through heterogeneous graph attention networks. IEEE Transactions on Neural Networks and Learning Systems 35, 4703–4717. doi:https://doi.org/10.1109/TNNLS.2021.3105617.

Hendrycks, D., Dietterich, T., 2019. Benchmarking neural network robustness to common corruptions and perturbations. CoRR doi:https://doi.org/10.48550/arXiv.1903.12261.

Hu, W., Tan, Y., 2022. Generating adversarial malware examples for black-box attacks based on gan, in: International Conference on Data Mining and Big Data, Springer Nature Singapore. pp. 409–423. doi:https://doi.org/10.1007/978-981-19-8991-9_29.

Huang, Y., Hu, H., Chen, C., 2021. Robustness of on-device models: Adversarial attack to deep learning models on android apps, in: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 101–110. doi:https://doi.org/10.1109/ICSE-SEIP52600.2021.00019.

Huda, S., Abawajy, J., Alazab, M., Abdollalihian, M., Islam, R., Yearwood, J., 2016. Hybrids of support vector machine wrapper and filter based framework for malware detection. Future Generation Computer Systems 55, 376–390. doi:https://doi.org/10.1016/j.future.2014.06.001.

Idika, M., 2007. A survey of malware detection techniques 48, 32–46.

Kabakus, A.T., 2022. Droidmalwaredetector: A novel android malware detection framework based on convolutional neural network. Expert Syst. Appl. 206. doi:https://doi.org/10.1016/j.eswa.2022.117833.

Khalid, H., Waseem, I.M., Muhammad, A., Xiangyong, L., Muhammad, A., 2023. Analysis of techniques for detection and removal of zero-day attacks (zda), in: Ubiquitous Security, Springer Nature Singapore. pp. 248–262. doi:https://doi.org/10.1007/978-981-99-0272-9_17.

Kim, J., Ban, Y., Ko, E., Cho, H., Yi, J.H., 2022. Mapas: a practical deep learning-based android malware detection system. Int. J. Inf. Secur. 21, 725–738. doi:https://doi.org/10.1007/s10207-022-00579-6.

Kim, J.Y., Bu, S.J., Cho, S.B., 2018. Zero-day malware detection using transferred generative adversarial networks based on deep autoencoders. Inf. Sci. 460, 83–102. doi:https://doi.org/10.1016/j.ins.2018.04.092.

Koli, J.D., 2018. Randroid: Android malware detection using random machine learning classifiers, in: 2018 Technologies for Smart-City Energy Security and Power (ICSESP), pp. 1–6. doi:https://doi.org/10.1109/ICSESP.2018.8376705.

Kucuk, Y., Yan, G., 2020. Deceiving portable executable malware classifiers into targeted misclassification with practical adversarial examples, in: Proceedings of the Tenth ACM Conference on Data and Application Security and Privacy, Association for Computing Machinery. p. 341–352. doi:https://doi.org/10.1145/3374664.3375741.

Langdon, W.B., Poli, R., 2013. Foundations of genetic programming. Springer Science & Business Media. doi:https://doi.org/10.1007/978-1-4614-6940-7_6.

Li, H., Zhou, S., Yuan, W., Luo, X., Gao, C., Chen, S., 2021. Robust android malware detection against adversarial example attacks, in: Proceedings of the Web Conference 2021, Association for Computing Machinery. p. 3603–3612. doi:https://doi.org/10.1145/3442381.3450044.

Li, L., Bartel, A., Bissyandé, T.F., Klein, J., Le Traon, Y., Arzt, S., Rasthofer, S., Bodden, E., Octeau, D., McDaniel, P., 2015. Iccta: Detecting inter-component privacy leaks in android apps, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, pp. 280–291. doi:https://doi.org/10.1109/ICSE.2015.48.

Li, S., Tang, Z., Li, H., Zhang, J., Wang, H., Wang, J., 2024a. Gmadv: An android malware variant generation and classification adversarial training framework. Journal of Information Security and Applications 84, 103800. doi:https://doi.org/10.1016/j.jisa.2024.103800.

Li, Y., Fang, S., Zhang, T., Cai, H., 2024b. Enhancing android malware detection: The influence of chatgpt on decision-centric task. arXiv e-prints doi:https://doi.org/10.48550/arXiv.2410.04352.

Liu, H., Sun, C., Su, Z., Jiang, Y., Gu, M., Sun, J., 2017. Stochastic optimization of program obfuscation, in: Proceedings of the 39th International Conference on Software Engineering, IEEE Press. p. 221–231. doi:https://doi.org/10.1109/ICSE.2017.28.

Liu, X., Du, X., Lei, Q., Liu, K., 2020. Multifamily classification of android malware with a fuzzy strategy to resist polymorphic familial variants. IEEE Access 8, 156900–156914. doi:https://doi.org/10.1109/ACCESS.2020.3019282.

Liu, X., Du, X., Zhang, X., Zhu, Q., Wang, H., Guizani, M., 2019. Adversarial samples on android malware detection systems for iot systems. Sensors 19. doi:https://doi.org/10.3390/s19040974.

Livshits, B., Whaley, J., Lam, M.S., 2005. Reflection analysis for java, in: Programming Languages and Systems, Springer Berlin Heidelberg. pp. 139–160. doi:https://doi.org/10.1007/11575467_11.

Lou, S., Cheng, S., Huang, J., Jiang, F., 2019. Tfdroid: Android malware detection by topics and sensitive data flows using machine learning techniques, in: 2019 IEEE 2nd International Conference on Information and Computer Technologies (ICICT), pp. 30–36. doi:https://doi.org/10.1109/INFOCT.2019.8711179.

Mahdavifar, S., Abdul Kadir, A.F., Fatemi, R., Alhadidi, D., Ghorbani, A.A., 2020. Dynamic android malware category classification using semi-supervised deep learning, in: 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech), pp. 515–522. doi:https://doi.org/10.1109/DASC-PICom-CBDCom-CyberSciTech49142.2020.00094.

Maiorca, D., Ariu, D., Corona, I., Aresu, M., Giacinto, G., 2015. Stealth attacks: An extended insight into the obfuscation effects on android malware. Computers & Security 51, 16–31. doi:https://doi.org/10.1016/j.cose.2015.02.007.

Mat, S.R.T., Razak, M.F.A., Kahar, M.N.M., Arif, J.M., Firdaus, A., 2022. A bayesian probability model for android malware detection. ICT Express 8, 424–431. doi:https://doi.org/10.1016/j.icte.2021.09.003.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Meng, G., Xue, Y., Mahinthan, C., Narayanan, A., Liu, Y., Zhang, J., Chen, T., 2016. Mystique: Evolving android malware for auditing anti-malware tools, in: Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security, Association for Computing Machinery. p. 365–376. doi:https://doi.org/10.1145/2897845.2897856.

Monnot, J., Toulouse, S., 2014. The traveling salesman problem and its variations. doi:https://doi.org/10.1002/9781119005353.ch7.

Nisan, N., Roughgarden, T., Éva Tardos, Vazirani, V.V. (Eds.), 2007. Algorithmic Game Theory. Cambridge University Press. doi:https://doi.org/10.1017/CBO9780511800481.

Octeau, D., Luchaup, D., Dering, M., Jha, S., McDaniel, P., 2015. Composite constant propagation: Application to android inter-component communication analysis, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, pp. 77–88. doi:https://doi.org/10.1109/ICSE.2015.30.

O'Shaughnessy, S., Sheridan, S., 2022. Image-based malware classification hybrid framework based on space-filling curves. Computers & Security 116, 102660. doi:https://doi.org/10.1016/j.cose.2022.102660.

Papernot, N., McDaniel, P., Jha, S., Fredrikson, M., Celik, Z.B., Swami, A., 2016. The limitations of deep learning in adversarial settings, in: 2016 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 372–387. doi:https://doi.org/10.1109/EuroSP.2016.36.

Pardalos, P.M., Mavridou, T.D., 2020. Simulated annealing, in: Encyclopedia of Optimization, Springer International Publishing. pp. 1–3. doi:https://doi.org/10.1007/978-3-030-54621-2_617-1.

Parikh, A.P., Täckström, O., Das, D., Uszkoreit, J., 2016. A decomposable attention model for natural language inference. ArXiv abs/1606.01933.

Pierazzi, F., Pendlebury, F., Cortellazzi, J., Cavallaro, L., 2020. Intriguing properties of adversarial ml attacks in the problem space, in: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1332–1349. doi:https://doi.org/10.1109/SP40000.2020.00073.

Qaisar, Z.H., Li, R., 2022. Multimodal information fusion for android malware detection using lazy learning. Multimedia Tools Appl. 81, 12077–12091. doi:https://doi.org/10.1007/s11042-021-10749-8.

Rabadi, D., Teo, S.G., 2020. Advanced windows methods on malware detection and classification, in: Proceedings of the 36th Annual Computer Security Applications Conference, Association for Computing Machinery. p. 54–68. doi:https://doi.org/10.1145/3427228.3427242.

Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., Nicholas, C.K., 2018. Malware detection by eating a whole exe, in: AAAI Workshops. doi:https://doi.org/10.48550/arXiv.1710.09435.

Raff, E., Fleshman, W., Zak, R., Anderson, H.S., Filar, B., McLean, M., 2021. Classifying sequences of extreme length with constant memory applied to malware detection. AAAI Conference on Artificial Intelligence 35, 9386–9394. doi:https://doi.org/10.1609/aaai.v35i11.17131.

Rastogi, V., Chen, Y., Jiang, X., 2014. Catch me if you can: Evaluating android anti-malware against transformation attacks. IEEE Transactions on Information Forensics and Security 9, 99–108. doi:https://doi.org/10.1109/TIFS.2013.2290431.

Rebuffi, S.A., Gowal, S., Calian, D., Stimberg, F., Wiles, O., Mann, T., 2021. Data augmentation can improve robustness, in: Proceedings of the 35th International Conference on Neural Information Processing Systems, Curran Associates Inc.. pp. 29935–29948. doi:https://doi.org/10.48550/arXiv.2111.05328.

Sagiv, M., Reps, T., Horwitz, S., 1996. Precise interprocedural dataflow analysis with applications to constant propagation. Theoretical Computer Science 167, 131–170. doi:10.1016/0304-3975(96)00072-2.

Samanta, P., Jain, S., 2024. Smarthash: Perceptual hashing for image tampering detection and authentication, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, Association for Computing Machinery. p. 1983–1993. doi:https://doi.org/10.1145/3627673.3679827.

Seungho, J., Moon, J., 2020. Malware-detection method with a convolutional recurrent neural network using opcode sequences. Information Sciences: An International Journal 535, 1–15. doi:https://doi.org/10.1016/j.ins.2020.05.026.

Shrivastava, G., Kumar, P., Gupta, D., Rodrigues, J.J.P.C., 2020. Privacy issues of android application permissions: A literature review. Trans. Emerg. Telecommun. Technol. 31, e3773. doi:https://doi.org/10.1002/ett.3773.

Sihag, V., Swami, A., Vardhan, M., Singh, P., 2020. Signature based malicious behavior detection in android, in: Computing Science, Communication and Security, Springer Singapore. pp. 251–262. doi:https://doi.org/10.1007/978-981-15-6648-6_20.

Singh, N., Tripathy, S., 2023. Mdldroid: Multimodal deep learning based android malware detection, in: Information Systems Security, Springer Nature Switzerland. pp. 159–177. doi:https://doi.org/10.1007/978-3-031-49099-6_10.

Sokolova, K., Perez, C., Lemercier, M., 2017. Android application classification and anomaly detection with graph-based permission patterns. Decis. Support Syst. 93, 62–76. doi:https://doi.org/10.1016/j.dss.2016.09.006.

Song, W., Li, X., Afroz, S., Garg, D., Kuznetsov, D., Yin, H., 2020. Mabmalware: A reinforcement learning framework for attacking static malware classifiers. ArXiv doi:https://doi.org/10.48550/arXiv.2003.03100.

Spreitzenbarth, M., Freiling, F., Echtler, F., Schreck, T., Hoffmann, J., 2013. Mobile-sandbox: having a deeper look into android applications, in: Proceedings of the 28th Annual ACM Symposium on Applied Computing, Association for Computing Machinery. p. 1808–1815. doi:https://doi.org/10.1145/2480362.2480701.

Suciu, O., Coull, S.E., Johns, J., 2019. Exploring adversarial examples in malware detection, in: 2019 IEEE Security and Privacy Workshops (SPW), pp. 8–14. doi:https://doi.org/10.1109/SPW.2019.00015.

Sun, S., Fu, X., Ruan, H., Du, X., Luo, B., Guizani, M., 2018. Real-time behavior analysis and identification for android application. IEEE Access 6, 38041–38051. doi:https://doi.org/10.1109/ACCESS.2018.2853121.

Sun, Y., Xie, Y., Qiu, Z., Pan, Y., Weng, J., Guo, S., 2017. Detecting android malware based on extreme learning machine, in: 2017 IEEE 15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech), pp. 47–53. doi:https://doi.org/10.1109/DASC-PICom-DataCom-CyberSciTec.2017.24.

Szegedy, C., Zaremba, W., Sutskever, I., Bruna, J., Erhan, D., Goodfellow, I.J., Fergus, R., 2013. Intriguing properties of neural networks. CoRR doi:https://doi.org/10.48550/arXiv.1312.6199.

Taylor, M.A., Larson, E.C., Thornton, M.A., 2021. Rapid ransomware detection through side channel exploitation, in: 2021 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 47–54. doi:https://doi.org/10.1109/CSR51186.2021.9527943.

Thangaveloo, R., Wang Jing, W., Kang Leng, C., Abdullah, J., 2020. Datdroid: Dynamic analysis technique in android malware detection. International Journal on Advanced Science, Engineering and Information Technology 10, 536–541. doi:https://doi.org/10.18517/ijaseit.10.2.10238.

Tumbleson, C., 2017. Apktool - a tool for reverse engineering 3rd party, closed, binary android apps. https://ibotpeaches.github.io/Apktool/. Accessed: Mar. 4, 2024.

Vasan, D., Alazab, M., Wassan, S., Safaei, B., Zheng, Q., 2020. Image-based malware classification using ensemble of cnn architectures (imcec). Computers & Security 92, 101748. doi:https://doi.org/10.1016/j.cose.2020.101748.

Wang, C., Xu, C., Yao, X., Tao, D., 2019. Evolutionary generative adversarial networks. IEEE Transactions on Evolutionary Computation 23, 921–934. doi:https://doi.org/10.1109/TEVC.2019.2895748.

Wang, C., Zhang, L., Zhao, K., Ding, X., Wang, X., 2021. Advandmal: Adversarial training for android malware detection and family classification. Symmetry 13. doi:https://doi.org/10.3390/sym13061081.

Wang, L., Wang, H., He, R., Tao, R., Meng, G., Luo, X., Liu, X., 2022. Malradar: Demystifying android malware in the new era. Proc. ACM Meas. Anal. Comput. Syst. 6, 1–27. doi:https://doi.org/10.1145/3530906.

Wang, Q., Hassan, W.U., Li, D., Jee, K., Yu, X., Zou, K., Rhee, J., Chen, Z., Cheng, W., Gunter, C.A., Chen, H., 2020. You are what you do: Hunting stealthy malware via data provenance analysis. Network and Distributed Systems Security (NDSS) , 23–26doi:https://dx.doi.org/10.14722/ndss.2020.24167.

Wang, W., Gao, Z., Zhao, M., Li, Y., Liu, J., Zhang, X., 2018. Droidensemble: Detecting android malicious applications with ensemble of string and structural static features. IEEE Access 6, 31798–31807. doi:https://doi.org/10.1109/ACCESS.2018.2835654.

Wu, Y., 2021. A survey on population-based meta-heuristic algorithms for motion planning of aircraft. Swarm and Evolutionary Computation 62, 100844. doi:https://doi.org/10.1016/j.swevo.2021.100844.

Wu, Y., Li, M., Zeng, Q., Yang, T., Wang, J., Fang, Z., Cheng, L., 2023. Droidrl:

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Feature selection for android malware detection with reinforcement learning. Computers & Security 128, 103126. doi:https://doi.org/10.1016/j.cose.2023.103126.

Xiong, S., Chen, X., Zhang, H., Wang, M., 2024. Domain adaptation-based deep learning framework for android malware detection across diverse distributions. Artificial Intelligence Advances 6, 13–24. doi:https://doi.org/10.30564/aia.v6i1.6718.

Xu, G., Xin, G., Jiao, L., Liu, J., Liu, S., Feng, M., Zheng, X., 2024. Ofei: A semi-black-box android adversarial sample attack framework against dlaas. IEEE Transactions on Computers 73, 956–969. doi:https://doi.org/10.1109/TC.2023.3236872.

Xu, W., Qi, Y., Evans, D., 2016. Automatically evading classifiers: A case study on pdf malware classifiers, in: Network and Distributed System Security Symposium. doi:http://dx.doi.org/10.14722/ndss.2016.23115.

Xue, Y., Meng, G., Liu, Y., Tan, T.H., Chen, H., Sun, J., Zhang, J., 2017. Auditing anti-malware tools by evolving android malware and dynamic loading technique. IEEE Transactions on Information Forensics and Security 12, 1529–1544. doi:https://doi.org/10.1109/TIFS.2017.2661723.

Yoo, S., Kim, S., Kim, S., Kang, B.B., 2021. Ai-hydra: Advanced hybrid approach using random forest and deep learning for malware classification. Information Sciences 546, 420–435. doi:https://doi.org/10.1016/j.ins.2020.08.082.

Yu, S., Wang, T., Wang, J., 2022. Data augmentation by program transformation. Journal of Systems and Software 190, 111304. doi:https://doi.org/10.1016/j.jss.2022.111304.

Zheng, J., Liu, J., Zhang, A., Zeng, J., Yang, Z., Liang, Z., Chua, T.S., 2024. Maskdroid: Robust android malware detection with masked graph representations, in: 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 331–343. doi:https://doi.org/10.1145/3691620.3695008.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

Appendix A. Sample codes for atomic mutations

Listing 2: Before Rename Identifier

```
.class public La/b/c/Class;

.method private p()V
// Body
.field private i:I
...
.end method
```

Listing 3: After Rename Identifier

```
// Rename Class Identifier
.class public La/b/c/Re;

// Rename Method Identifier
.method private q()V
// Body
// Rename Field Identifier
.field private j:I
...
.end method
```

Listing 4: Before Rename Resource

```
// In the file directory:
.\res\drawable\bg.png

// In <item> of bg.xml:
android:drawable="@drawable/bg"
```

Listing 5: After Rename Resource

```
// In the file directory:
.\res\drawable\re.png

// In <item> of bg.xml:
android:drawable="@drawable/re"
```

Listing 6: Before Flow Obfuscation

```
invoke-direct {p0}, La/b/c;-><init>()V
new-instance v0, Ljava/util/ArrayList;
```

Listing 7: After Flow Obfuscation

```
goto :goto_0
new-instance v0, Ljava/util/ArrayList;

:goto_0
invoke-direct {p0}, La/b/c;-><init>()V
```

Listing 8: Before Encode Data

```
invoke-virtual {v1}, La/b/c;->p()Ljava/lang/
    String;
move-result-object v2
```

Listing 9: After Encode Data

```
// Invoke Encode
invoke-direct {v2, v2}, La/b/c;->encodeP(
    Ljava/lang/String;)Ljava/lang/String;
move-result-object v2

// Use XOR Based Algorithms to Encrypt
.method private stringEncode(Ljava/lang/
    String;)Ljava/lang/String;
.locals 5
.param p1, "str"    # Ljava/lang/String;

.prologue
const-string v0, "key"

invoke-virtual {p1}, Ljava/lang/String;->
    toCharArray()[C
move-result-object v0

invoke-virtual {p1}, Ljava/lang/String;->
    length()I
move-result v4

new-array v1, v4, [C
move-object v2, v1

const/4 v3, 0x0

:goto_0
if-ge v3, v4, :cond_0

aget-char v1, v0, v3
xor-int/lit8 v1, v1, 0x41

aput-char v1, v2, v3
add-int/lit8 v3, v3, 0x1

goto :goto_0
:cond_0
new-instance v1, Ljava/lang/String;
invoke-direct {v1, v2}, Ljava/lang/String;-><
    init>([C)V
return-object v1
.end method
```

Listing 10: Before Reflection

```
invoke-static {v1, p0}, La/b/c;->p(ILandroid/
    content/Context;)Ljava/lang/String;
move-result-object v1
```

Listing 11: After Reflection

```
const-class v2, La/b/c;

const-string v3, "p"

const/16 v4, 0x2
new-array v4, v4, [Ljava/lang/Class;

const/4 v5, 0x0

const-class v6, Ljava/lang/Integer;
aput-object v6, v4, v5

const/4 v5, 0x1

const-class v6, Landroid/content/Context;
aput-object v6, v4, v5

invoke-virtual {v2, v3, v4}, Ljava/lang/Class
    ;->getDeclaredMethod(Ljava/lang/String;[
    Ljava/lang/Class;)Ljava/lang/reflect/
    Method;
move-result-object v3
```


Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094

```
    const/4 v4, 0x1
    invoke-virtual {v3, v4}, Ljava/lang/reflect/
        Method;->setAccessible(Z)V

    const/4 v4, 0x0

    invoke-virtual {v3, v4, v1, p0}, Ljava/lang/
        reflect/Method;->invoke(Ljava/lang/Object
        ;[Ljava/lang/Object;)Ljava/lang/Object;
    move-result-object v1
```

Listing 12: Before Indirect Method Call

```
    invoke-direct {p0}, La/b/c;->p()V
```

Listing 13: After Indirect Method Call

```
    invoke-direct {p0}, La/b/c;->q()V

    .method private q()V
    .locals 0

    .prologue
    invoke-direct {p0}, La/b/c;->p()V
    .end method
```

Listing 14: Before Replace API

```
    invoke-static {v1, v2}, Landroid/util/Log;->d
        (Ljava/lang/String;Ljava/lang/String;)I
```

Listing 15: After Replace API

```
    invoke-static {v1, v2}, Landroid/util/Log;->v
        (Ljava/lang/String;Ljava/lang/String;)I
```

Listing 16: Before API downgrade

```
    // In API 28:
    invoke-static {v1, v2, v3, v4}, Landroid/
        system/Os;->sendfile(Ljava/io/
        FileDescription;Ljava/io/FileDescription;
        Landroid/system/Int64Ref;J;)J
```

Listing 17: After API downgrade

```
    // In API 27:
    invoke-static {v1, v2, v3, v4}, Landroid/
        system/Os;->sendfile(Ljava/io/
        FileDescription;Ljava/io/FileDescription;
        Landroid/util/MutableLong;J;)J
```

Listing 18: Before Reduce API Level

```
    sdkInfo:
    minSdkVersion: '14'
```

Listing 19: After Reduce API Level

```
    sdkInfo:
    minSdkVersion: '11'
```

Listing 20: After Add Permission

```
    <uses-permission android:name="android.
        permission.BLUETOOTH"/>
```

Listing 21: After Insert API snippets

```
    invoke-virtual {}, Landroid/location/Location
        ;->getTime()Ljava/lang/String;
```

Listing 22: After Add Permission & API

```
    // Add Permission in AndroidManifest.xml:
    <uses-permission android:name="android.
        permission.SEND_SMS"/>

    // Add API in the Selected SMALI File:
    invoke-direct {p0}, La/b/c;->sendSMS(Landroid
        /content/Context;)V
```


Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5146094